

Powerfactory FMI

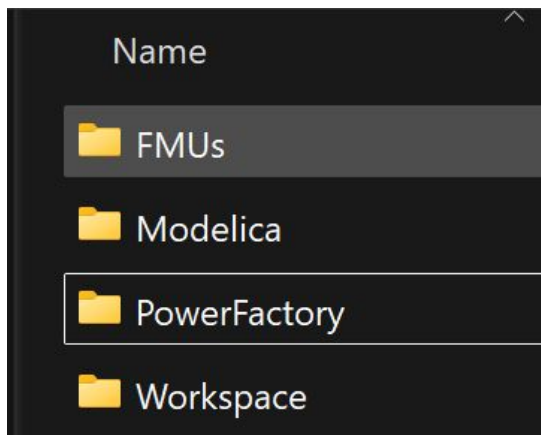
Part I: OpenModelica FMU SCRX Export

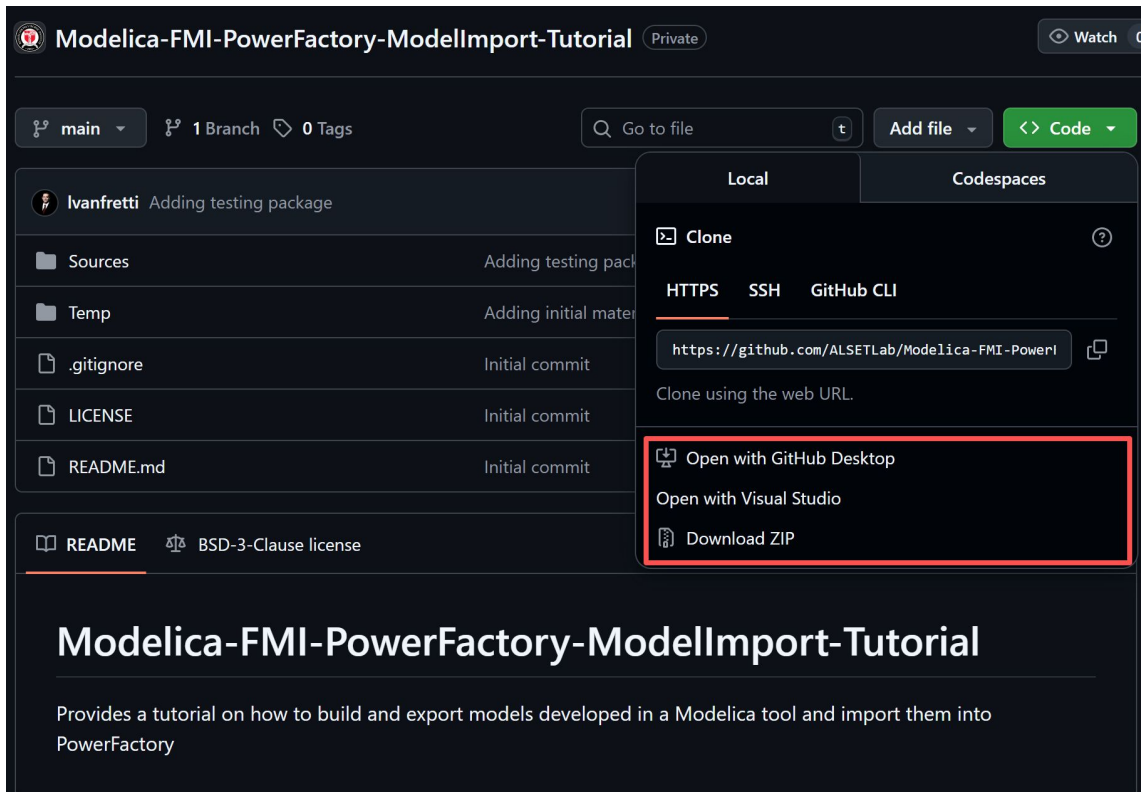


This page serves as the **cover and introduction** to the tutorial. It visually highlights the connection between **PowerFactory (PF)** and the **Functional Mock-up Interface (FMI)** standard. The icons at the top indicate the tools we will use, while the file list below ([SCRX.fmu](#), [SCRX.mo](#), and [scrxGridFMITest.pfd](#)) represents the core components of the example project.

The description should clarify for the reader:

- This tutorial will walk through how to use FMI models within PowerFactory.
- The example files shown here (FMU package, Modelica model, and PowerFactory project file) will be used in later steps.
- By the end, readers will understand how to **import, configure, and simulate FMI models in PowerFactory**.





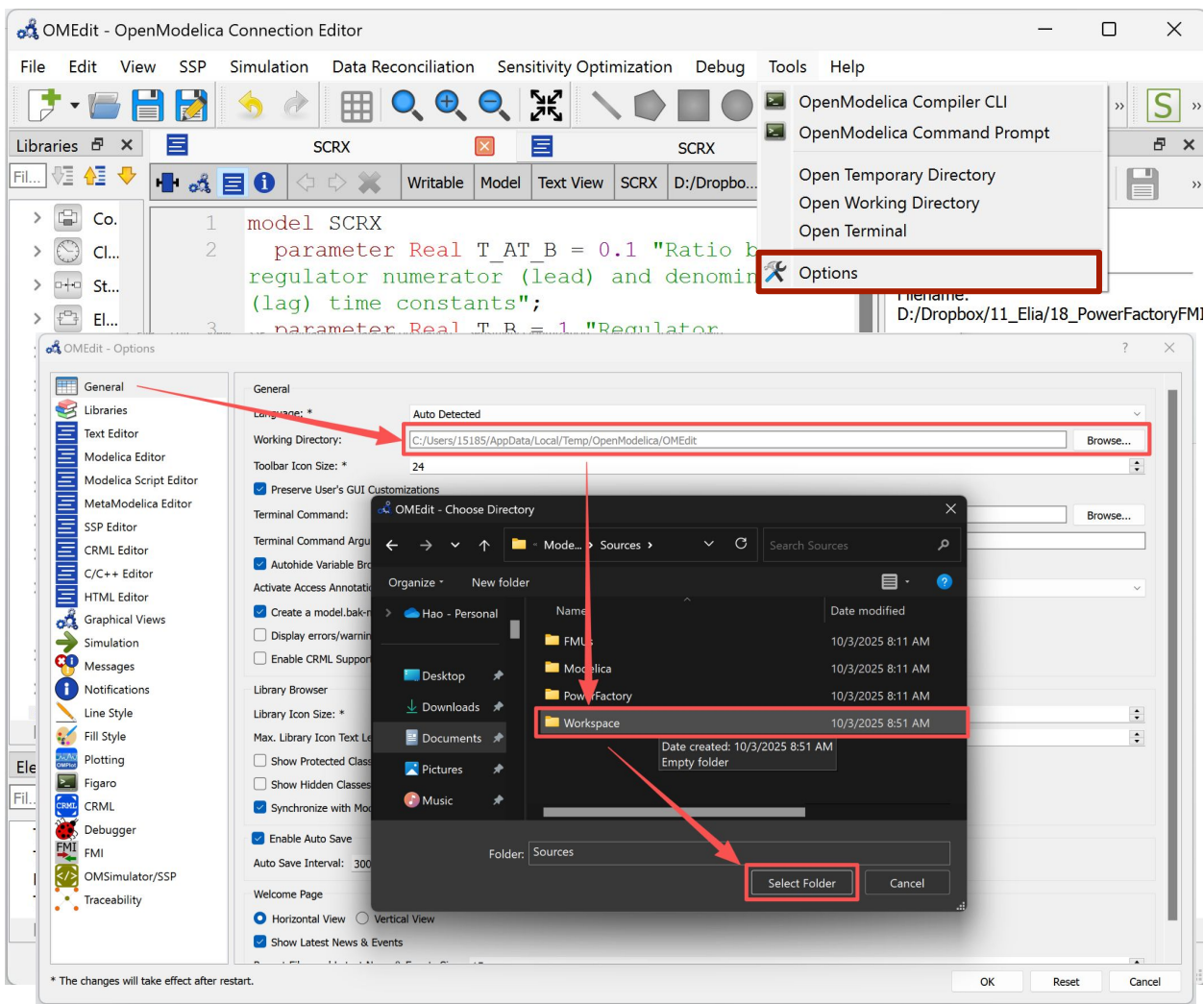
Before beginning the import process, you need the example files prepared for this tutorial. These files are hosted on GitHub under the repository:

Modelica-FMI-PowerFactory-ModelImport-Tutorial

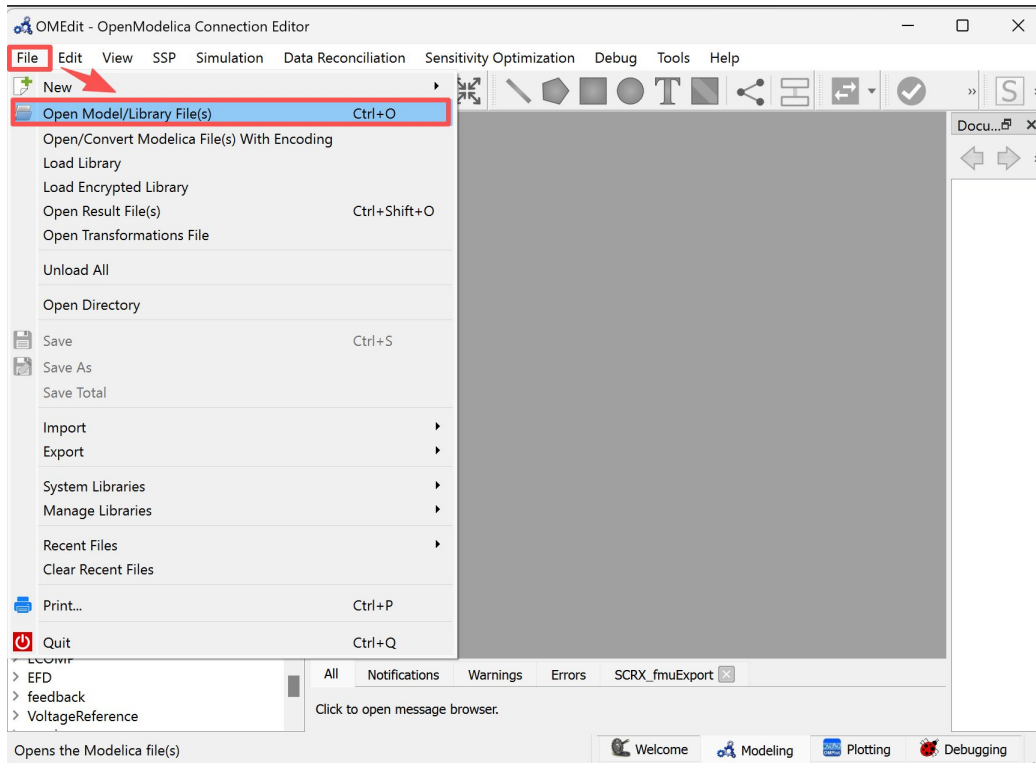
<https://github.com/ALSETLab/Modelica-FMI-PowerFactory-ModelImport-Tutorial>

To get the files:

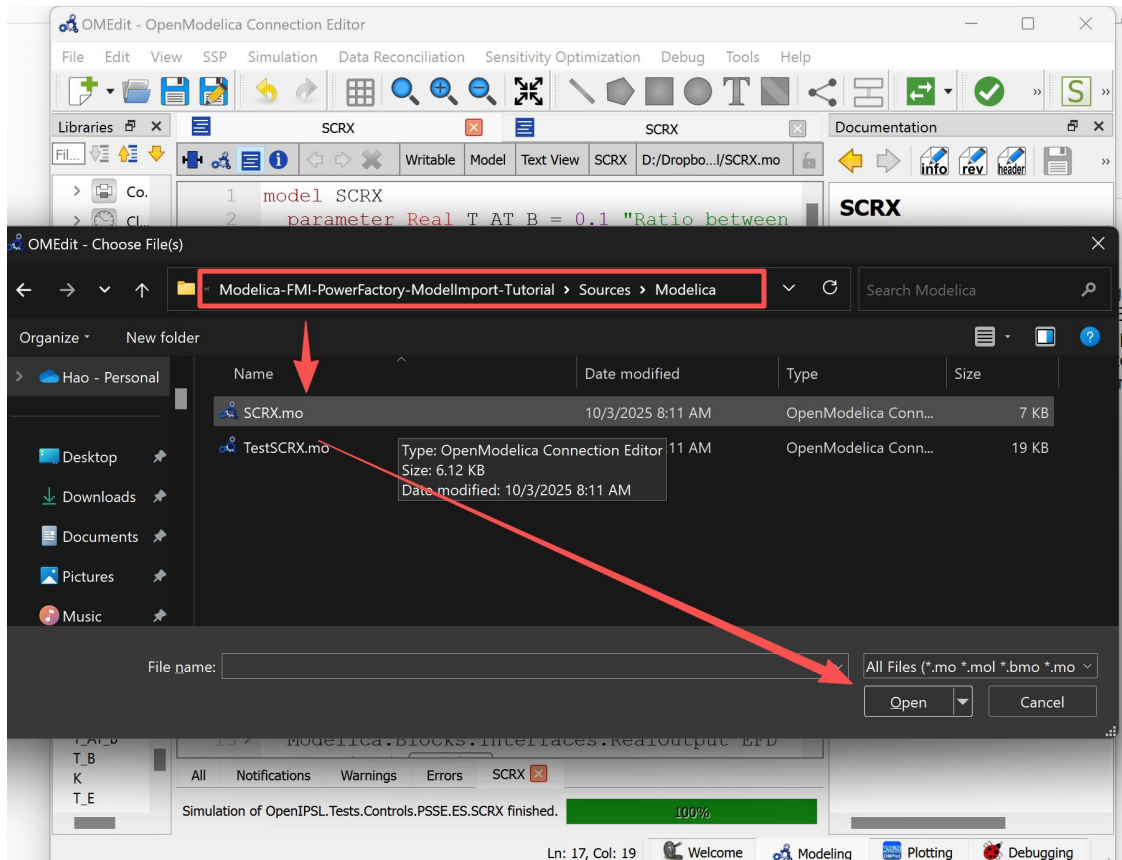
1. Navigate to the GitHub repository page.
2. Click on the green **Code** button.
3. Choose one of the following options:
 - **Open with GitHub Desktop** (recommended if you use GitHub Desktop for version control).
 - **Open with Visual Studio** (if working within Visual Studio).
 - **Download ZIP** (simplest option, extracts the files locally without Git).



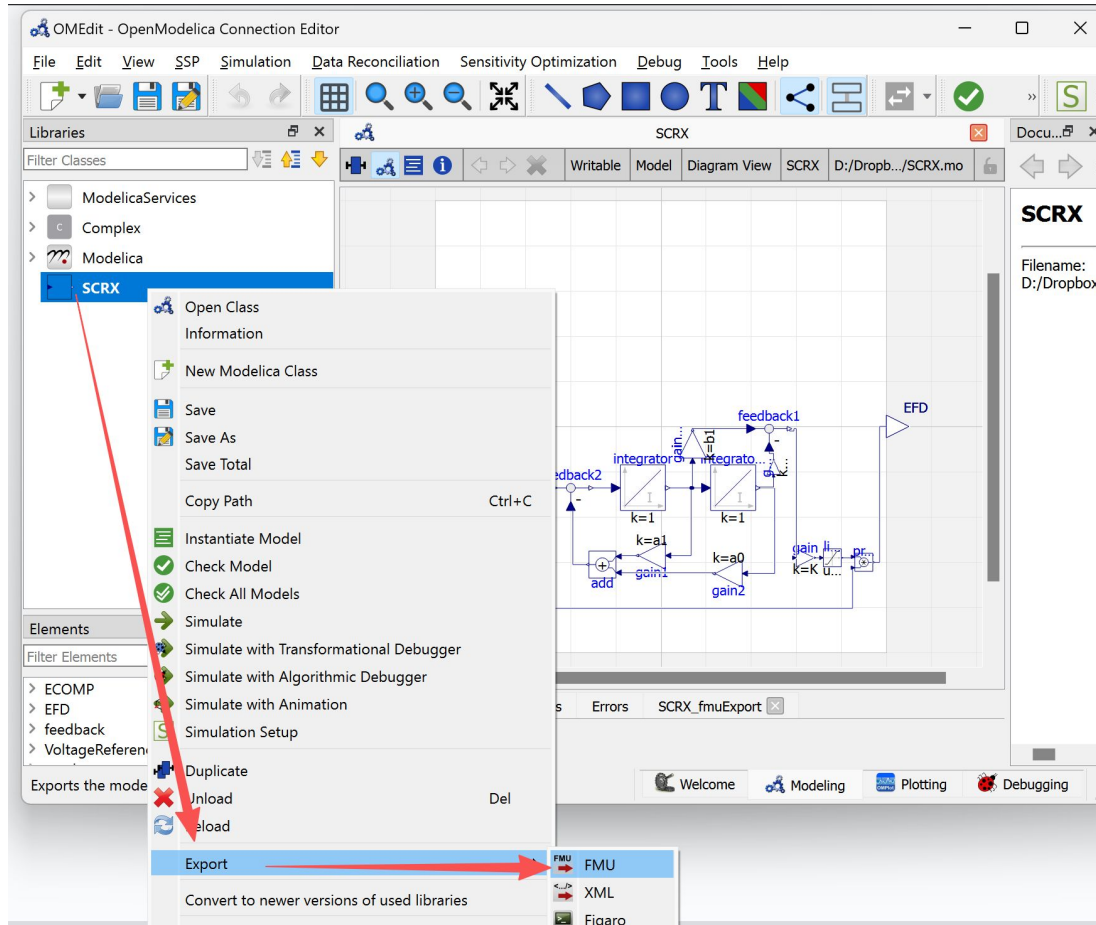
1. Open **OMEdit** (OpenModelica Connection Editor).
2. Navigate to **Tools** → **Options**.
3. In the **General** tab, locate the **Working Directory** field.
4. Click **Browse** and select your desired folder. For this tutorial, create or select a workspace folder (e.g., **Workspace**) in github repository.
5. Press **Select Folder**, then click **OK** in the Options window.



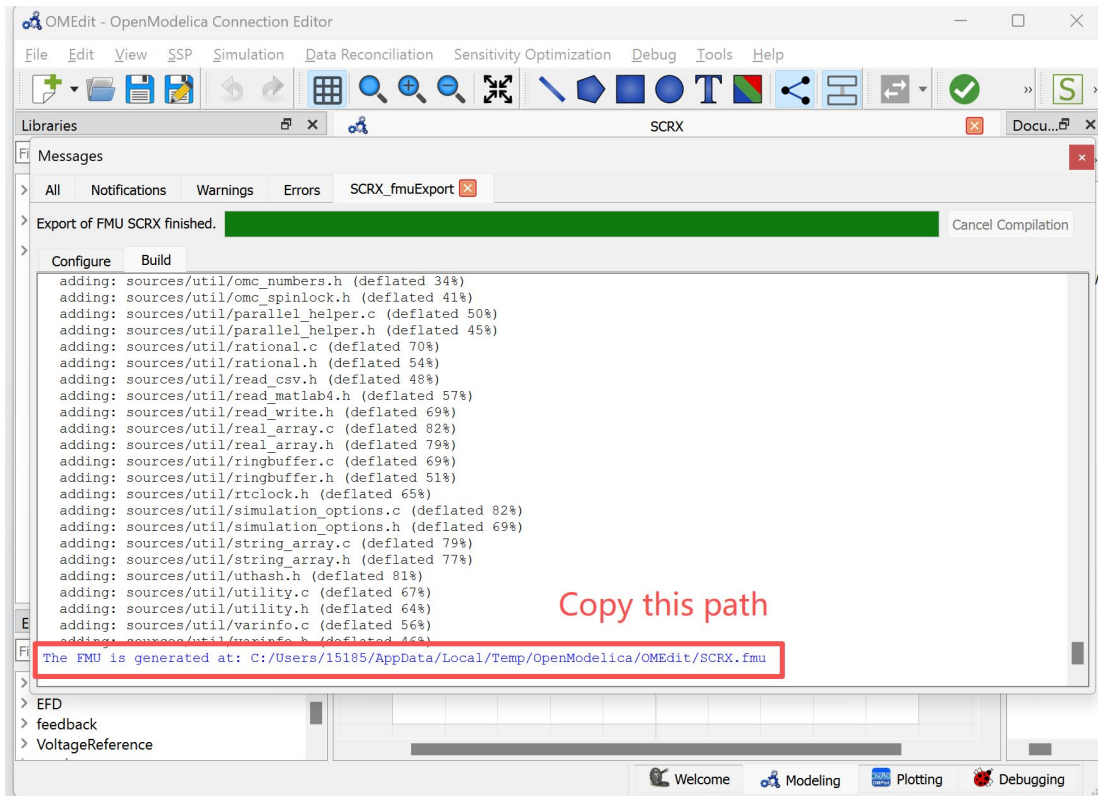
This page shows how to begin working with the Modelica file that will later be exported as an FMU for use in PowerFactory. In **OMEdit (OpenModelica Connection Editor)**, navigate to **File** → **Open Model/Library File(s)** (or use the shortcut **Ctrl+O**) to load the **.mo** file.



After choosing **File** → **Open Model/Library File(s)** in OMEdit, this step shows how to locate and load the specific Modelica model that will be used in the tutorial. In this case, the file **SCRX.mo** is selected from the project directory.

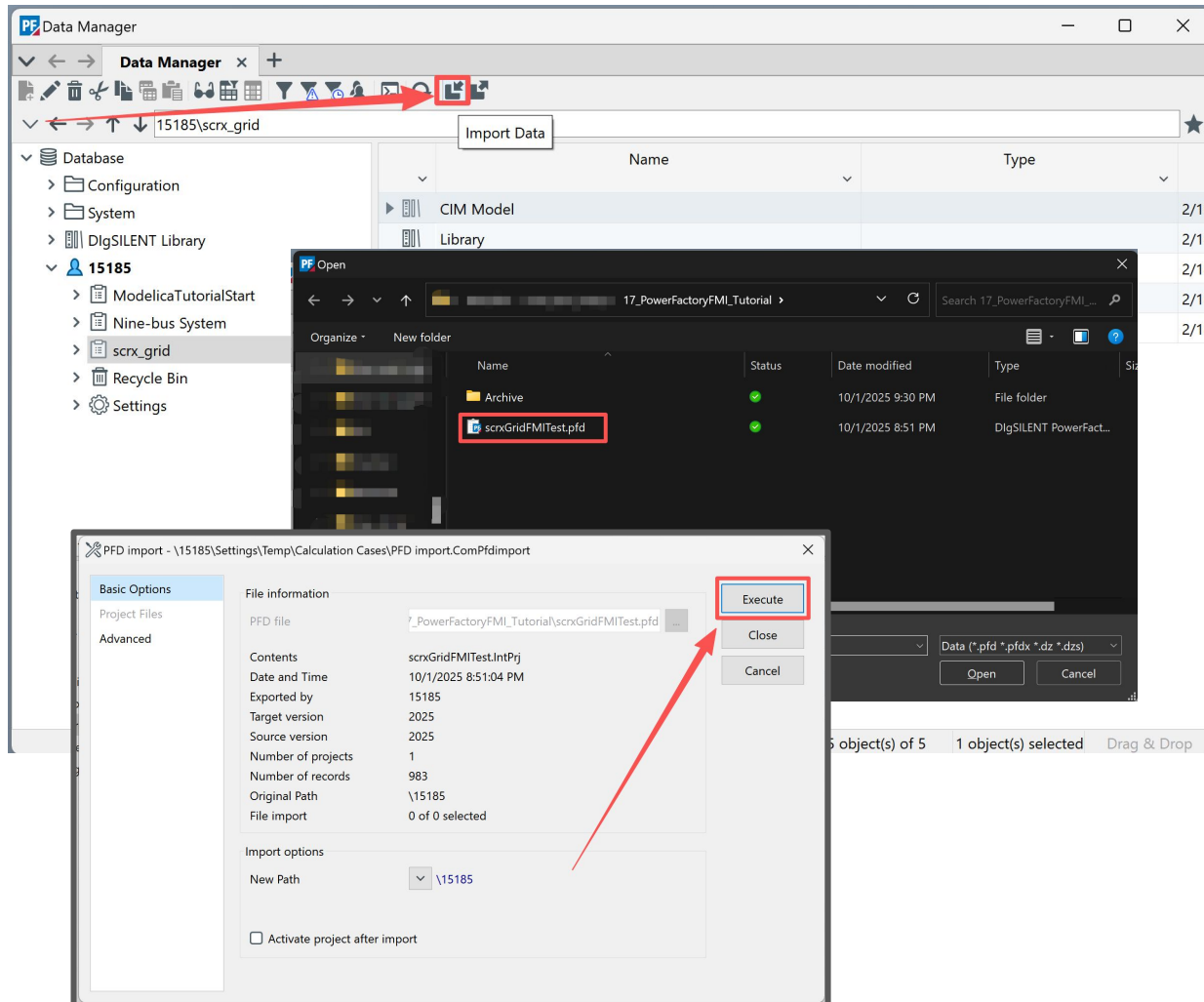


With the Modelica file loaded, the next step is to export it as a **Functional Mock-up Unit (FMU)**. In OMEdit, right-click on the model (**SCRX** in this case), then select **Export** → **FMU** from the context menu.



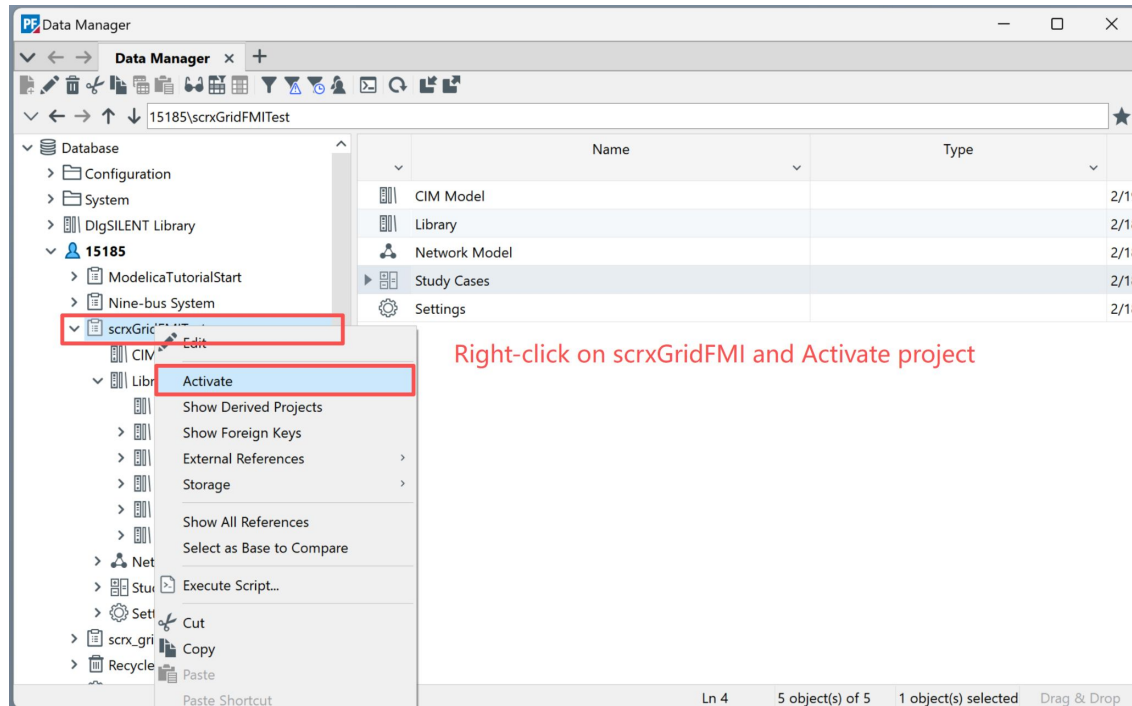
Once the export process finishes, OMEdit confirms that the FMU has been successfully generated. A green progress bar indicates completion, and the **output log** provides the location of the exported file.

It is important to **copy this path** or move the FMU file to your working directory for easier access later. This FMU package will be imported into PowerFactory in the next steps of the tutorial.



In PowerFactory's Data Manager, create or select a database folder (here named 15185/scrx_grid) and choose Import Data. From the file selection window, locate the provided project file scrxGridFMITest.pfd.

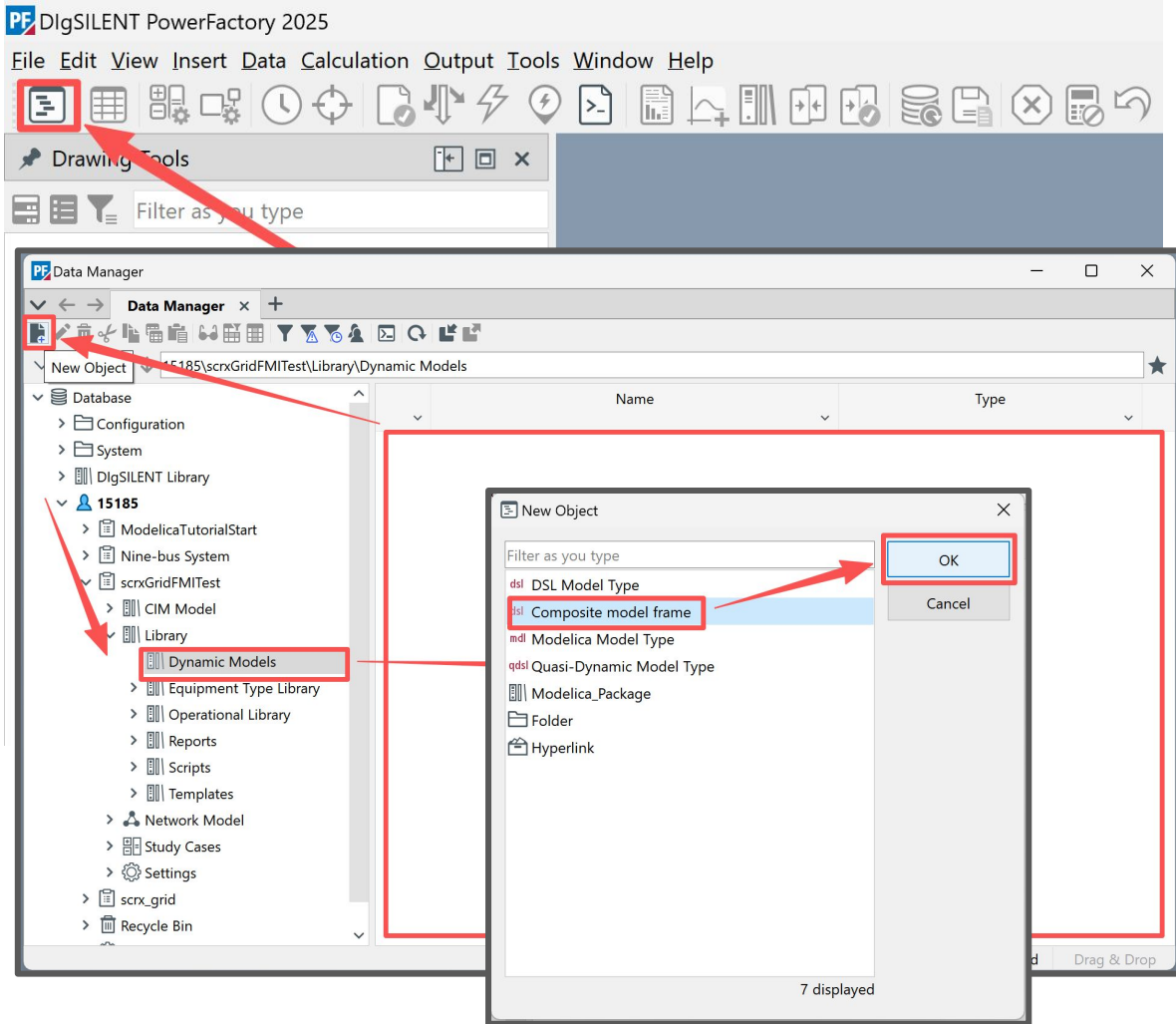
After selecting the file, confirm the import options in the PFD Import dialog and click Execute to load the project. Once imported, the case will appear in the Data Manager and can be activated for further work.



Right-click on scrxGridFMI and Activate project

In PowerFactory's Data Manager, create or select a database folder (here named 15185/scrx_grid) and choose Import Data. From the file selection window, locate the provided project file scrxGridFMITest.pfd.

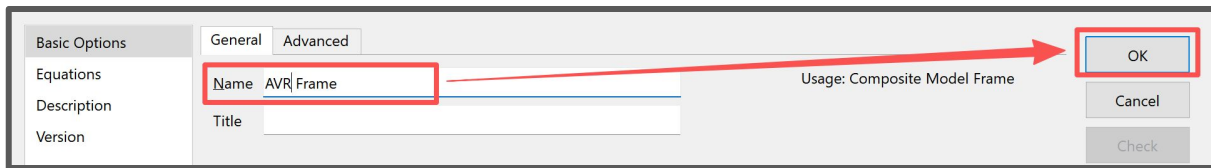
After selecting the file, confirm the import options in the PFD Import dialog and click Execute to load the project. Once imported, the case will appear in the Data Manager and can be activated for further work.



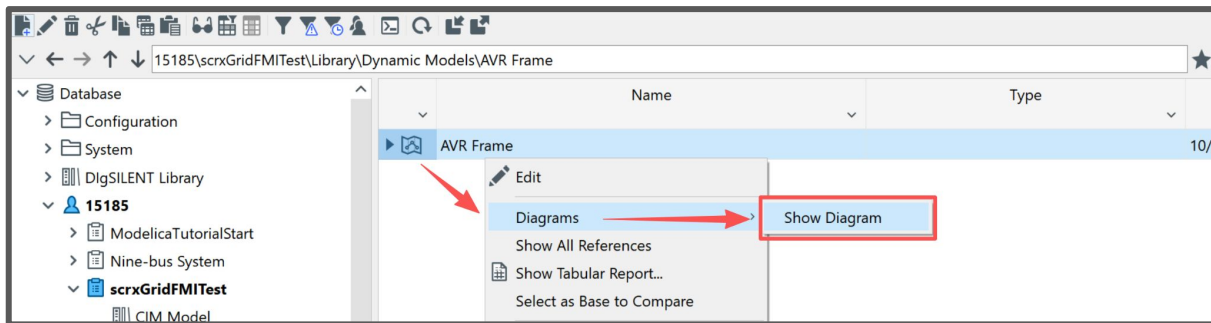
To prepare PowerFactory for integrating the FMU, create a new Composite Model Frame inside the project's Dynamic Models library.

1. In the Data Manager, navigate to: Library → Dynamic Models
2. Click the New Object button in the toolbar.
3. In the New Object dialog, select Composite model frame and click OK.

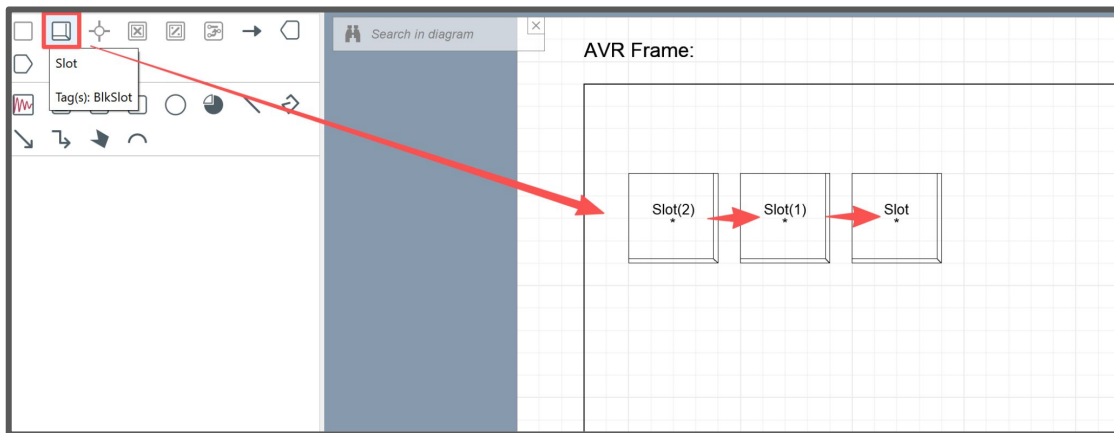
The Composite Model Frame acts as a container that will later be linked to the FMU and mapped to elements in the network model. This step establishes the foundation for embedding external FMI-based models into PowerFactory's dynamic simulation environment.



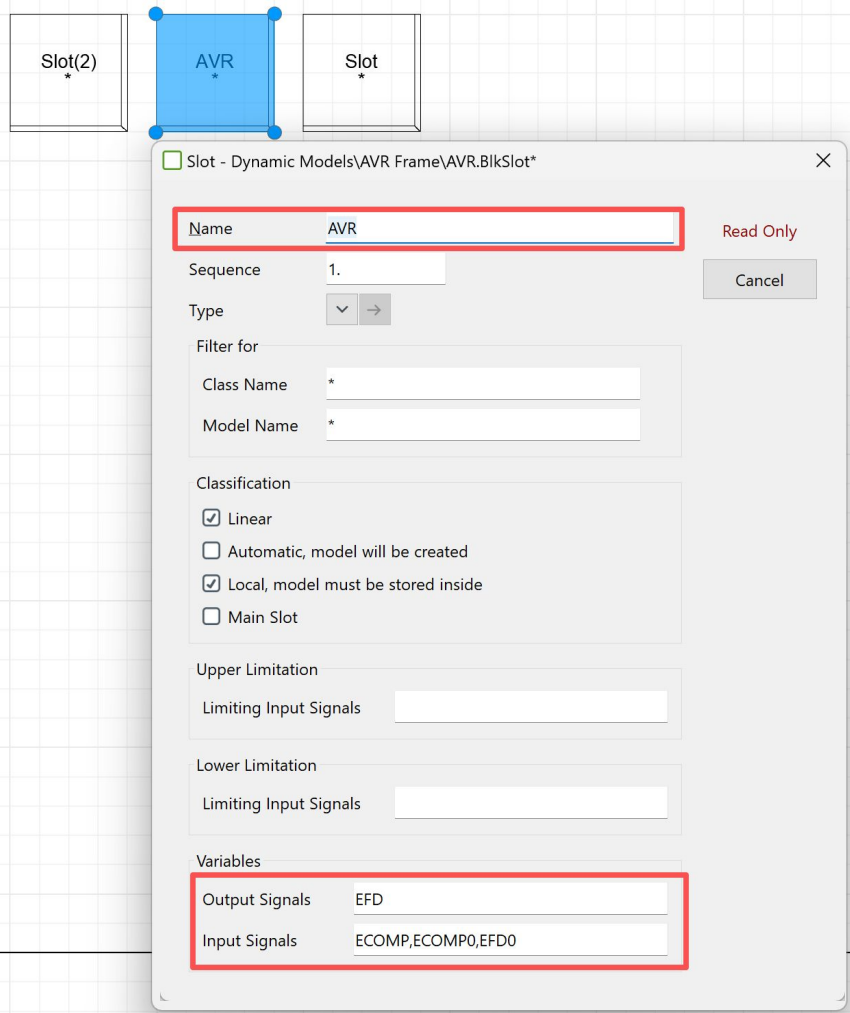
After creating the Composite Model Frame, give it a clear name (here: AVR Frame) and confirm with OK. This creates a new dynamic model container inside the project library.



1. Right-click on the new object (AVR Frame) and select Show Diagram.
2. In the diagram editor, add Slots from the toolbar. Slots act as placeholders for model connections (inputs/outputs) and will later be mapped to the FMU interface.
3. Arrange the slots in sequence to represent the expected signal flow.



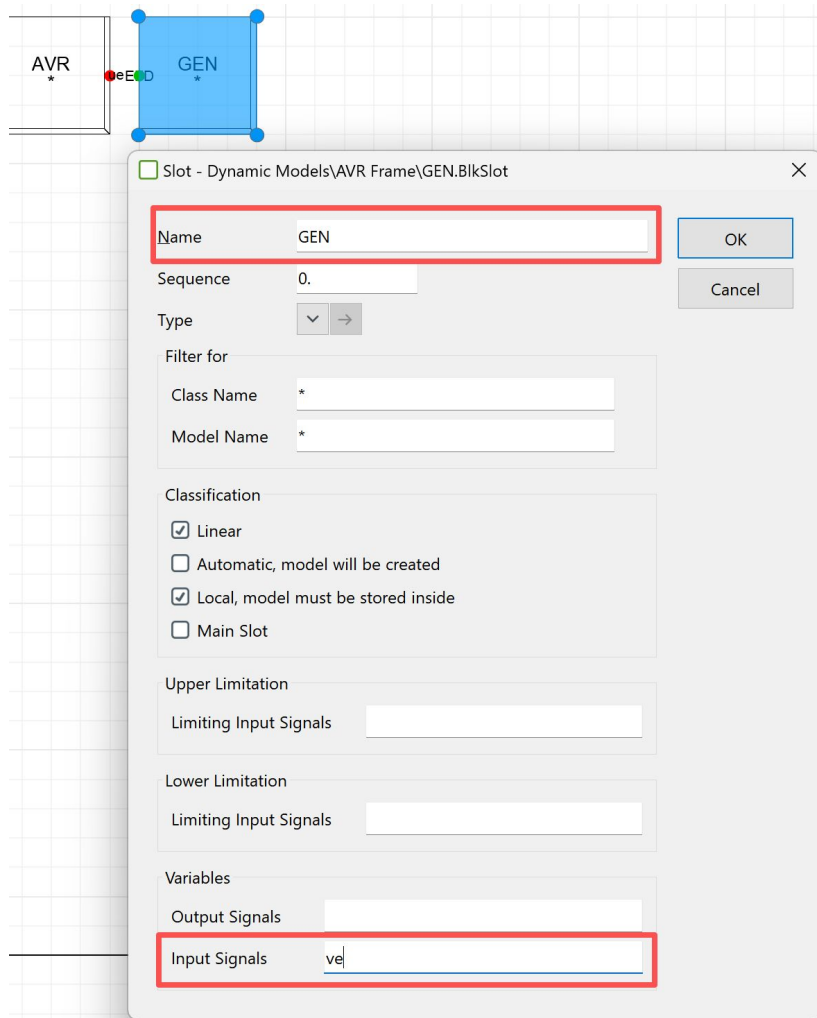
This step establishes the framework where the FMU will be integrated, ensuring that the signals can be correctly routed between the FMU and PowerFactory's simulation environment.



Inside the **Composite Model Frame** diagram, each slot must be configured to represent the correct role in the control structure.

1. Double-click on the slot and assign it a **Name** (here: *AVR*).
2. Define the **Input Signals** and **Output Signals**:
 - **Input Signal**: *ECOMP, ECOMP0, EFD0*
 - **Output Signal**: *EFD*
3. Keep relevant options selected (e.g., *Linear*) to ensure proper signal handling.

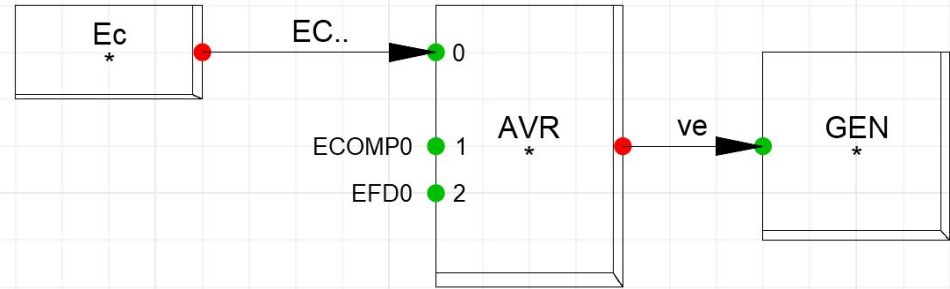
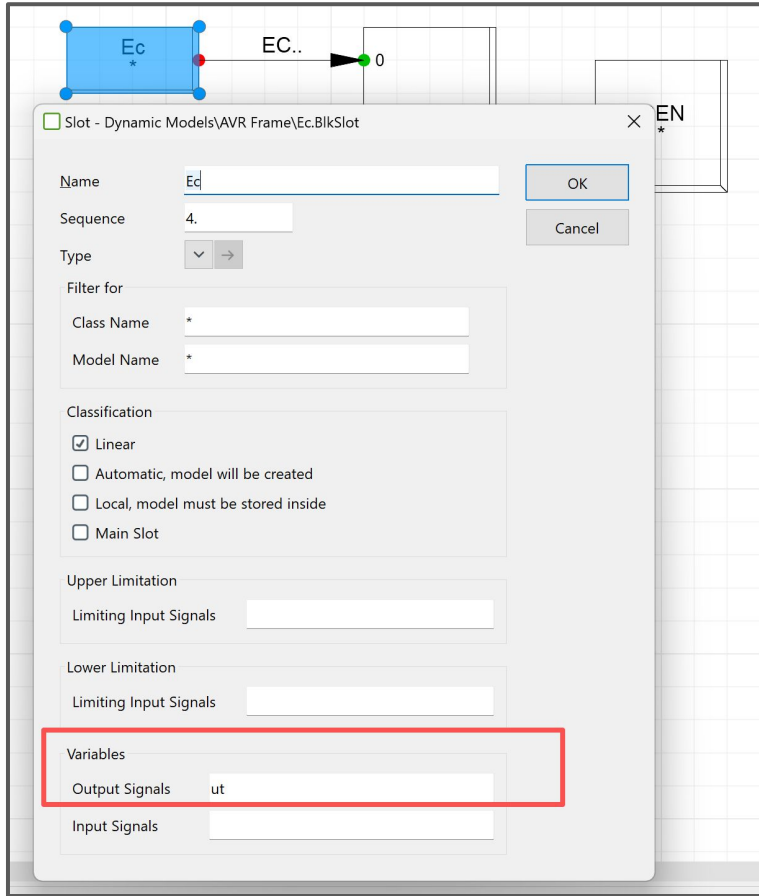
This step links the abstract slot object to the intended control variables, enabling PowerFactory to map them to FMU ports later. Properly naming and assigning inputs/outputs is critical for ensuring the FMI model integrates smoothly into the grid simulation.



In addition to the AVR slot, a **Generator (GEN) slot** is created in the Composite Model Frame to represent the machine interface.

1. Double-click the new slot and set its **Name** to **GEN**.
2. Define the **Input Signals** (here: **ve**, representing the voltage input).
3. Keep the relevant classification settings (e.g., *Linear*) enabled.

This slot establishes the connection point for the generator within the control loop. By combining the **AVR slot** (controller) with the **GEN slot** (plant), the composite model can exchange signals with the FMU in a structured way, ensuring proper integration during simulation.



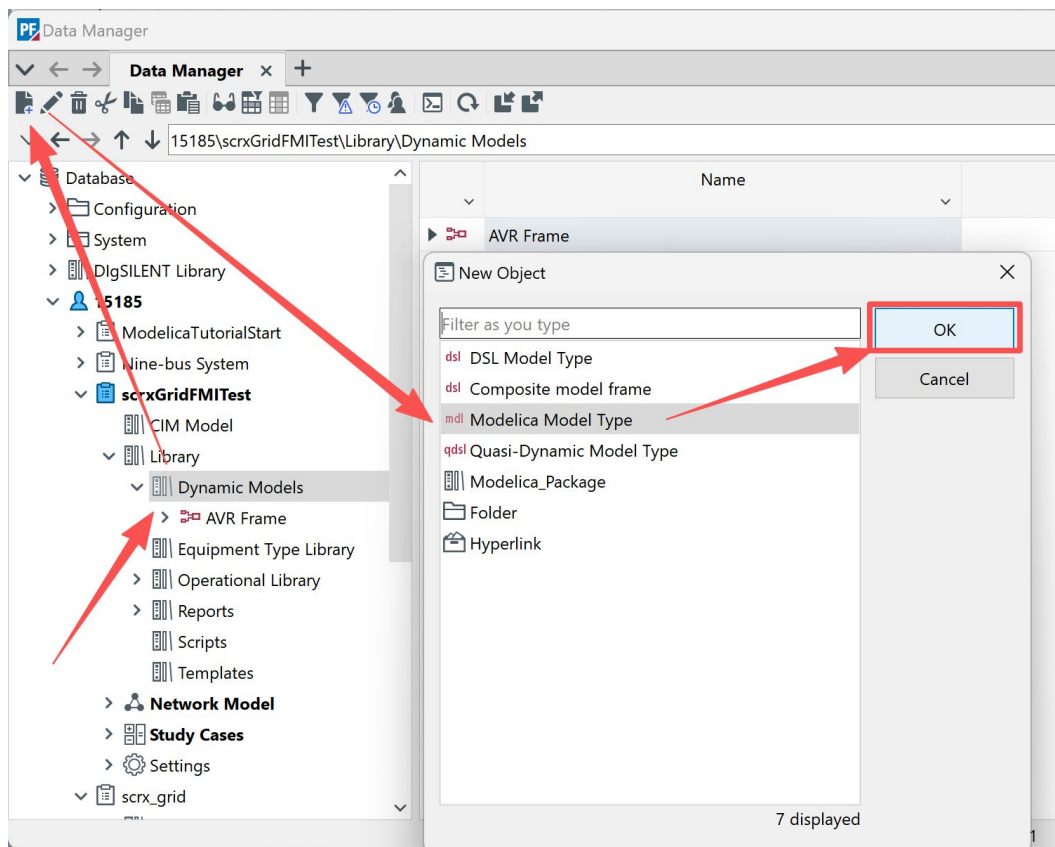
The last slot to configure in the composite model is the Ec (excitation control input).

1. Double-click the slot and set the Name to Ec
2. Define the Output Signals (here: u).
3. Ensure the classification options remain consistent with the other slots (e.g., Linear).

With the ECOMP slot added, the signal flow is now complete:

- ECOMP → AVR → GEN

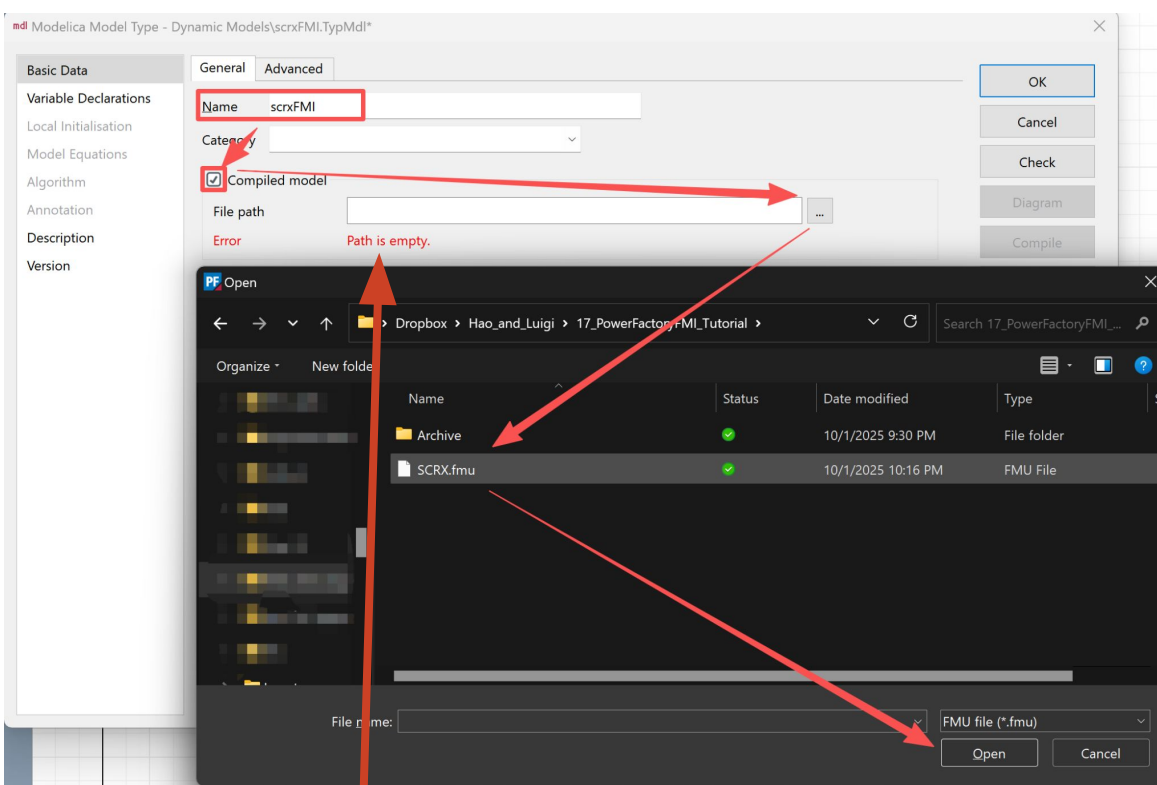
This structure mirrors the logical flow of the excitation system: Ec provides the input, the AVR processes and regulates the signal, and the GEN slot represents the generator receiving the final control input.



With the Composite Model Frame prepared, the next step is to add a **Modelica Model Type** inside the **Dynamic Models** library.

1. In the **Data Manager**, navigate to:
Library → Dynamic Models
2. Click **New Object** from the toolbar.
3. In the dialog, select **Modelica Model Type** and click **OK**.

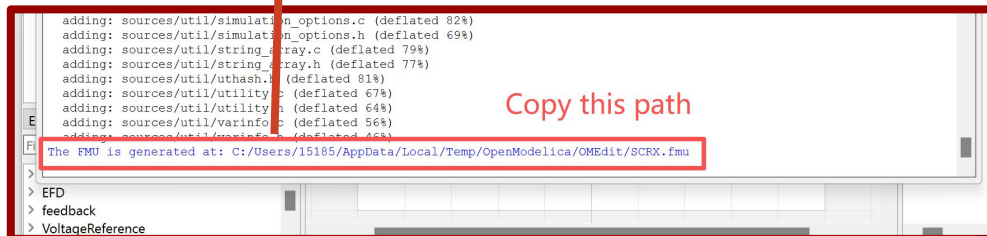
The Modelica Model Type object serves as the actual container for importing and linking the FMU into PowerFactory. Later, this object will be referenced inside the Composite Frame to bind the FMU model with the defined slots and signal connections.

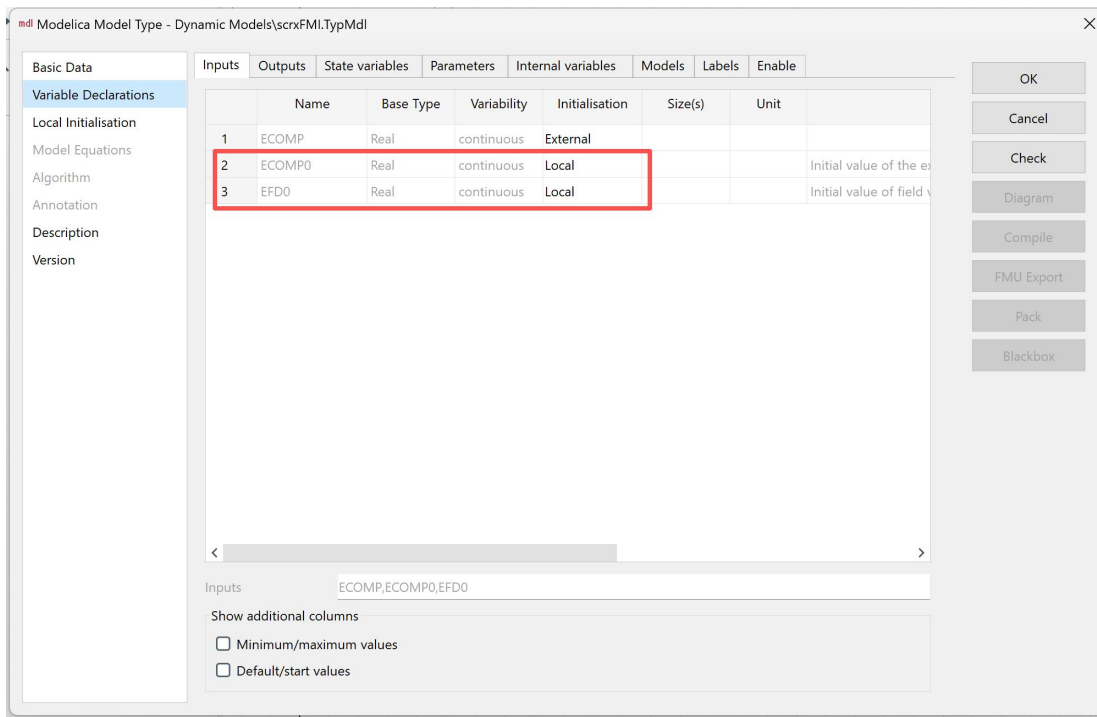


In the **Modelica Model Type** dialog, configure the object to use the exported FMU:

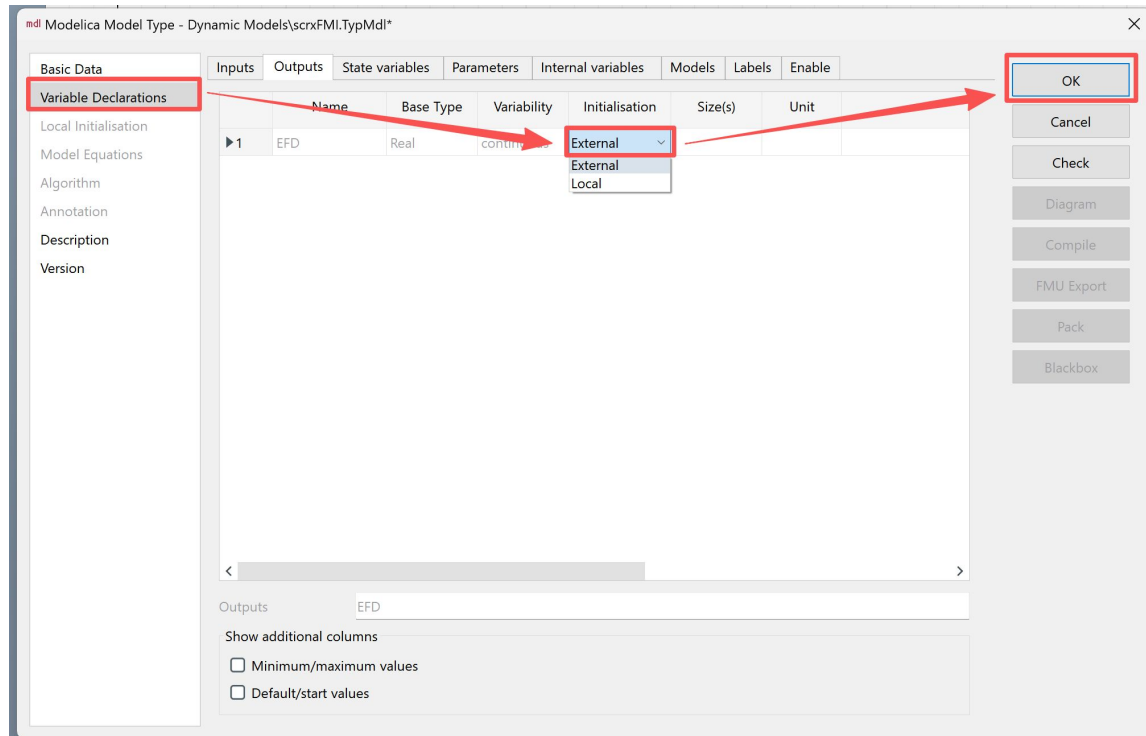
1. Enter a descriptive **Name** (e.g., *scrxFMI*).
2. Check **Compiled model** to indicate that this object references an external FMU rather than native Modelica code.
3. Under **File path**, browse to the FMU generated earlier (e.g., *SCRX.fmu*). You can also copy the path directly from the OpenModelica export log.

This step establishes the formal link between PowerFactory and the external FMU. Once saved, this Modelica Model Type can be assigned to slots in the Composite Model Frame, enabling the FMU to exchange signals with the network model.



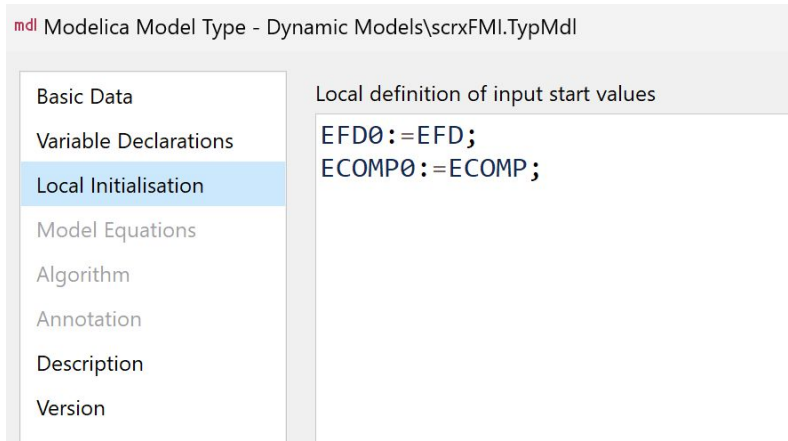


1. **Load Flow Execution:**
PowerFactory calculates the steady-state values for all components in the system.
2. **Variable Mapping:**
The Local Initialization section of the Model Type defines how those values are mapped to the FMU's inputs
3. **FMU Start-Up:**
When the simulation begins, PowerFactory automatically passes these initialized values through the input ports so that the FMU starts with the same steady-state as the PowerFactory network.



1. Open the **Variable Declarations** tab.
2. Add the FMU variables (e.g., *EFD* as an output).
3. Set the **Initialisation** field to **External**, indicating that this variable will be provided by PowerFactory rather than defined locally.
4. Confirm by clicking **OK**.

Declaring variables as *External* ensures that PowerFactory correctly maps inputs and outputs between the FMU and the composite frame slots. This step is essential for signal exchange during dynamic simulations.



Go to Local Initialisation

In the left panel, select **Local Initialisation**.

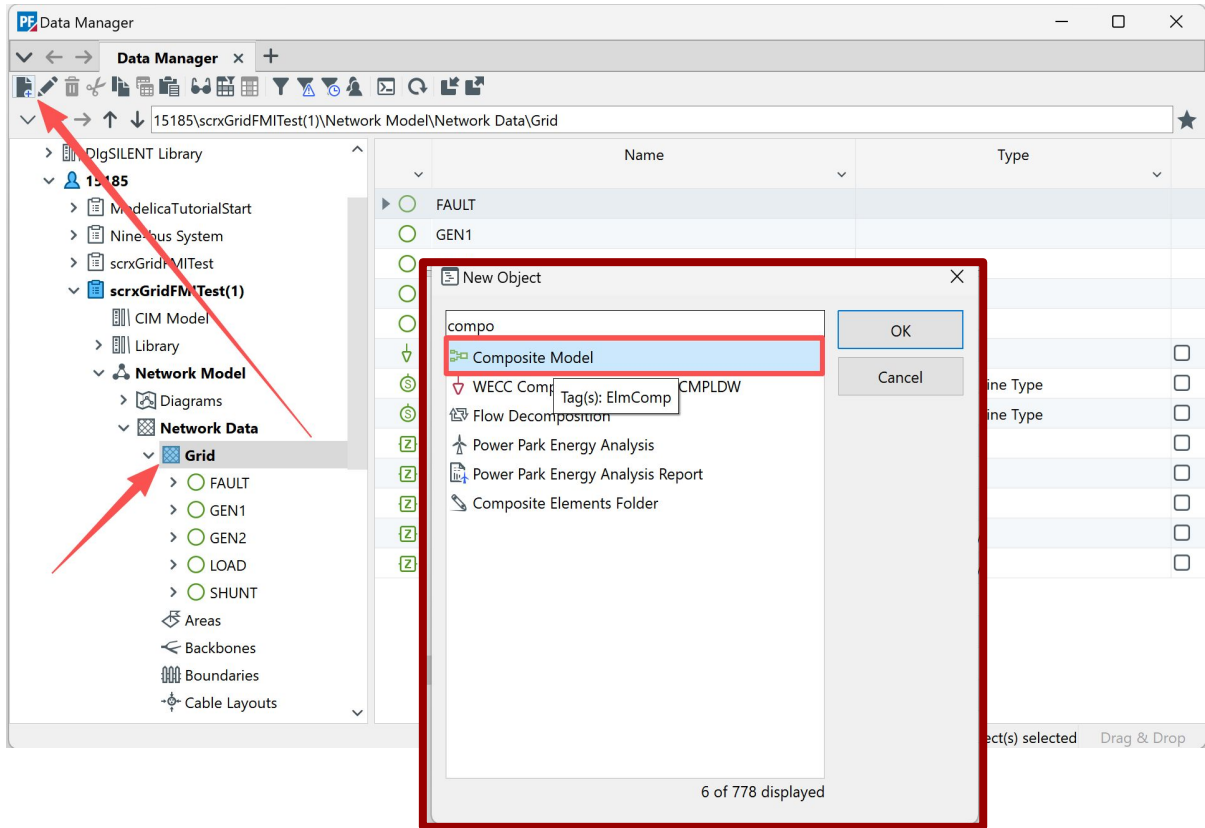
This section allows you to specify how PowerFactory initializes the FMU input ports with values from the system load flow.

```
EFD0:=EFD;
```

```
ECOMP0:=ECOMP;
```

Define Input Start Values

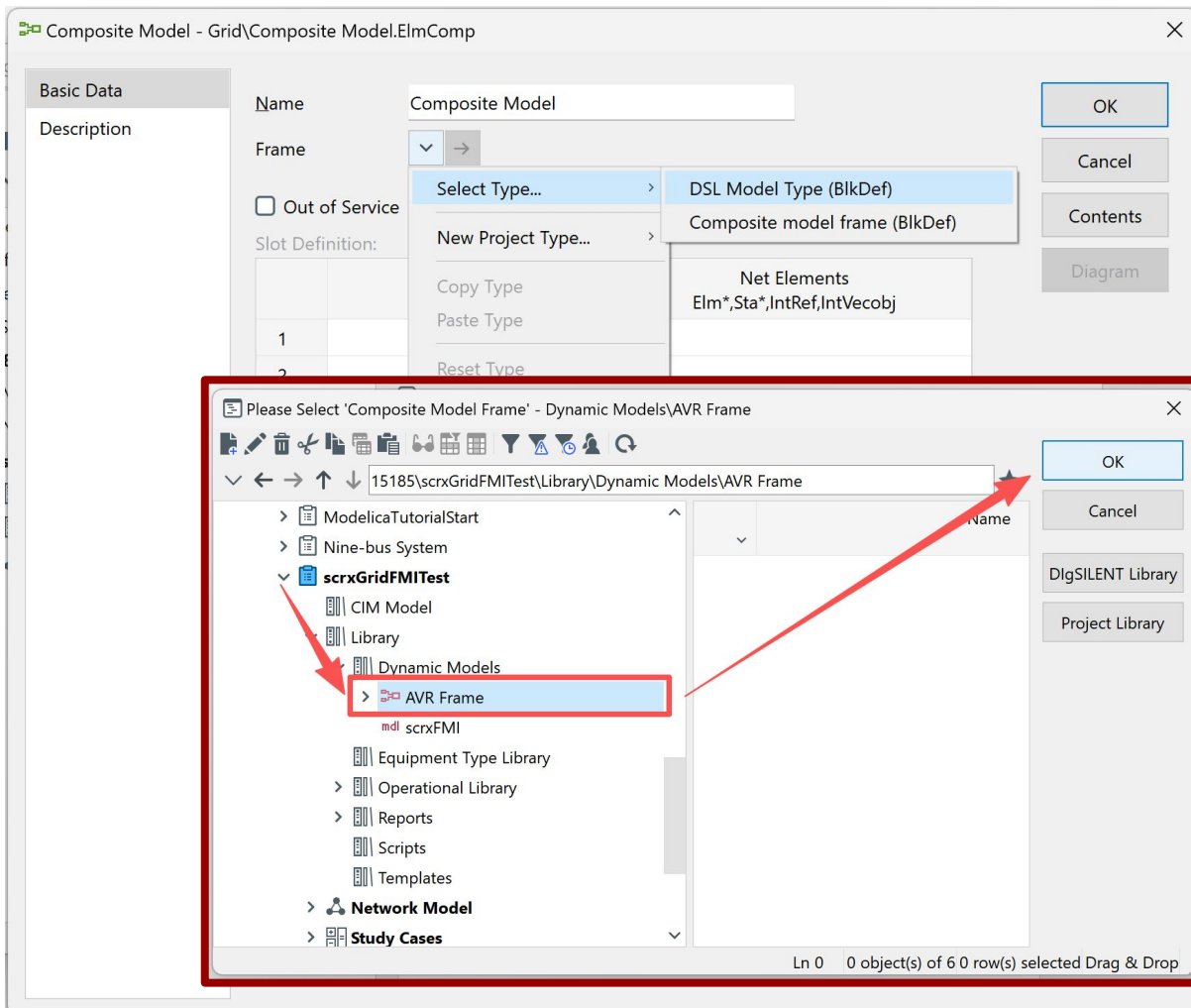
Enter the following assignments under *Local definition of input start values*:



To integrate the FMU-based model into the grid, create a **Composite Model** object in the **Network Data** section:

1. In the **Data Manager**, navigate to:
Network Model → **Network Data** → **Grid**.
2. Click the **New Object** button.
3. From the dialog, select **Composite Model** and confirm with **OK**.

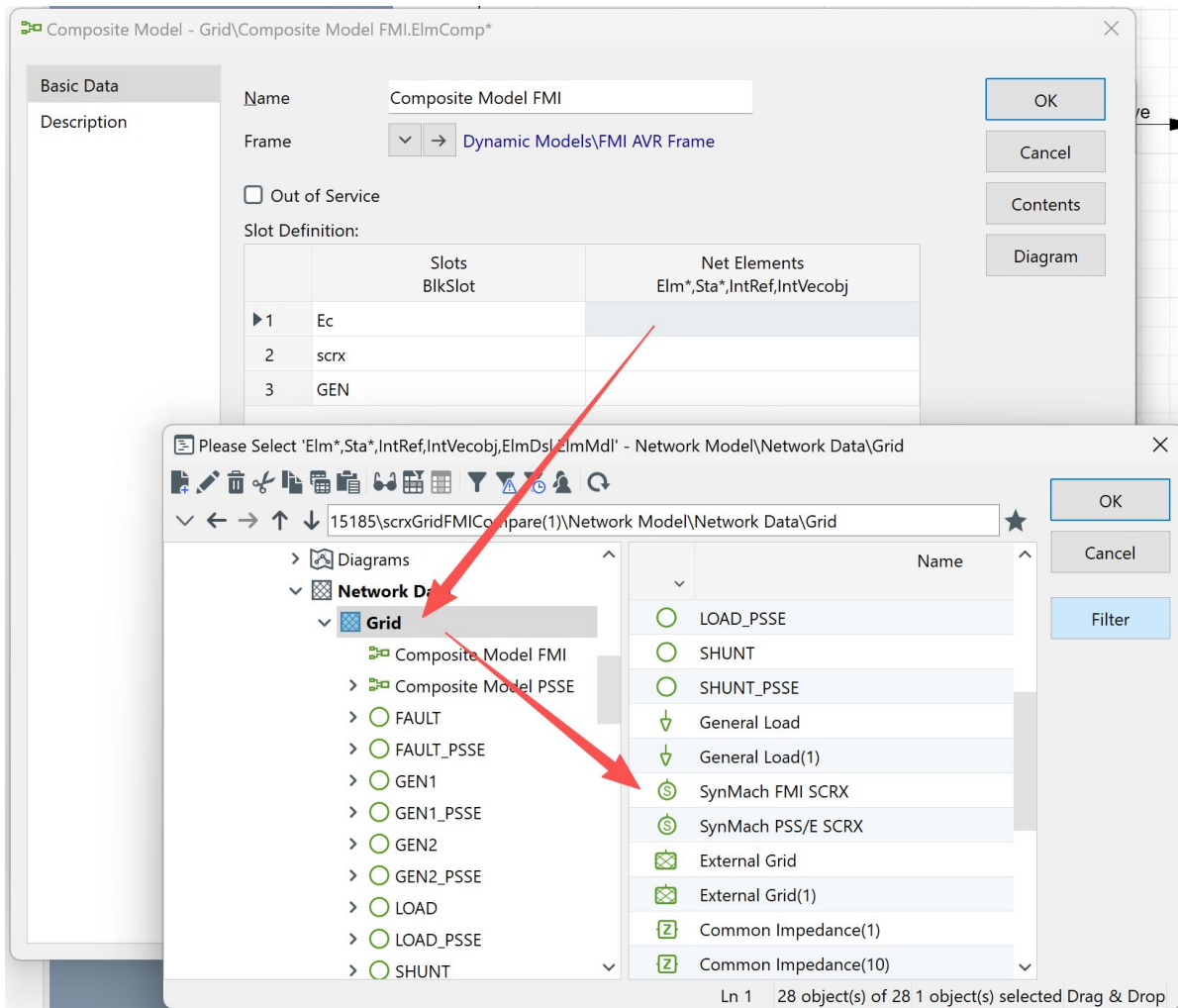
The Composite Model created here acts as the actual network element that will reference the **Composite Model Frame** and the **FMU-backed Modelica Model Type** defined earlier. This step bridges the gap between the abstract dynamic model library and the physical grid structure.



After creating the **Composite Model** in the grid, you need to link it to the previously defined **Composite Model Frame**.

1. In the Composite Model properties, select **Frame** → **Composite model frame (BlkDef)**.
2. Browse through the library and select the correct frame (here: *AVR Frame*).
3. Confirm with **OK**.

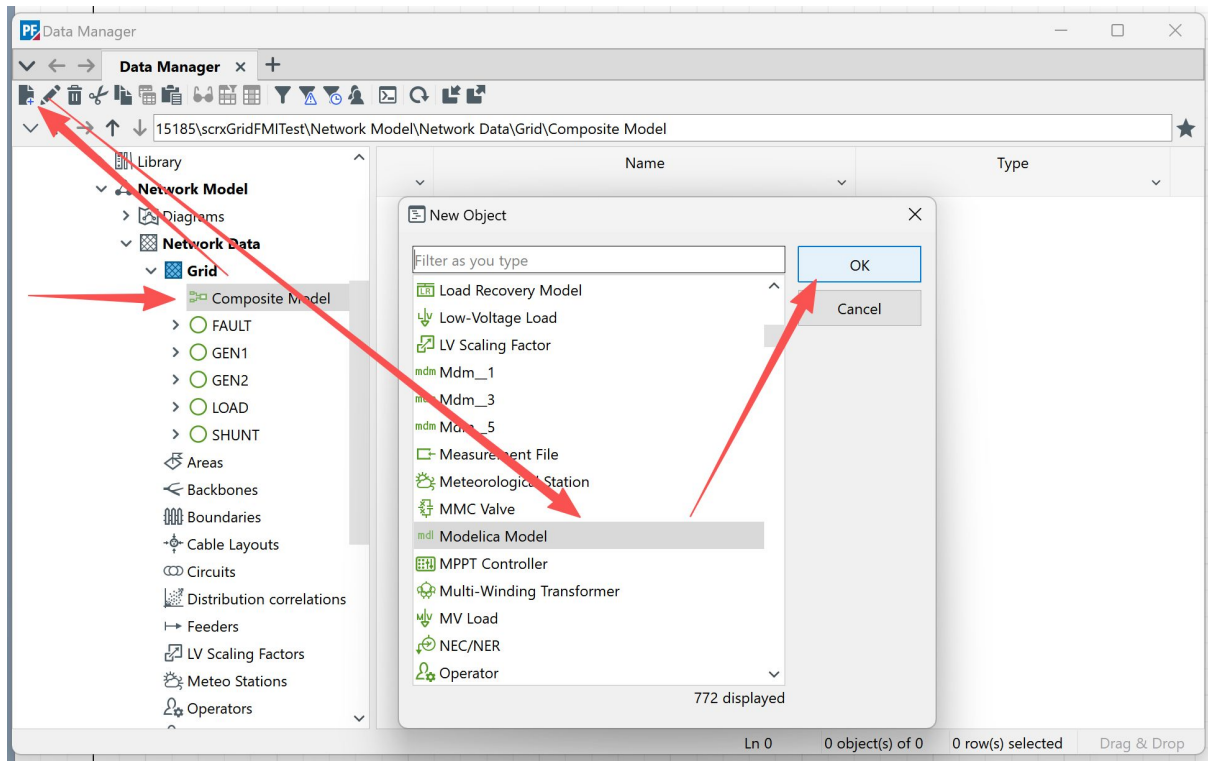
By assigning the Composite Model Frame, the network-level object is now associated with the slot structure you designed earlier. This ensures that signals exchanged in the simulation will follow the defined slot connections and map correctly to the FMU.



After assigning the **Composite Model Frame**, the next step is to map each slot to the corresponding element in the network.

1. In the **Composite Model** properties, select the slot (e.g., *GEN*).
2. From the **Net Elements** list, choose the associated grid element — in this case, a **Synchronous Machine FMI**.
3. Confirm with **OK**.

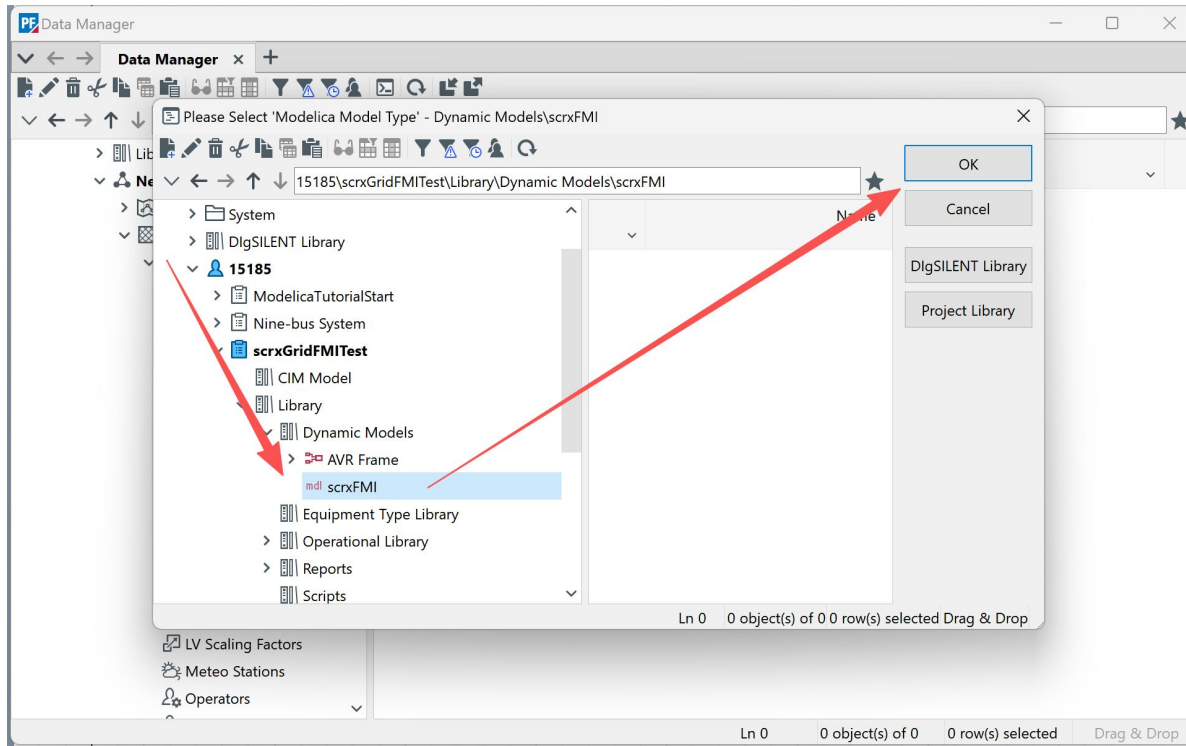
This mapping links the slot placeholders defined in the composite frame (e.g., AVR, GEN, ECOMP) to the actual components in the PowerFactory network model. With this step, the FMI model is fully integrated into the simulation environment and ready to exchange signals with real grid elements.



With the **Composite Model** created and mapped to the grid, the next step is to insert the FMU-backed Modelica object.

1. In the **Composite Model** under *Network Data* → *Grid*, click **New Object**.
2. From the dialog, select **Modelica Model** and confirm with **OK**.

This Modelica Model object represents the FMU within the network-level composite model. It provides the actual implementation that will interact with the slots (AVR, GEN, ECOMP) defined earlier, completing the connection between the FMU and the grid simulation environment.



Once the **Modelica Model** object is created inside the Composite Model, you need to assign it to the correct **Modelica Model Type**.

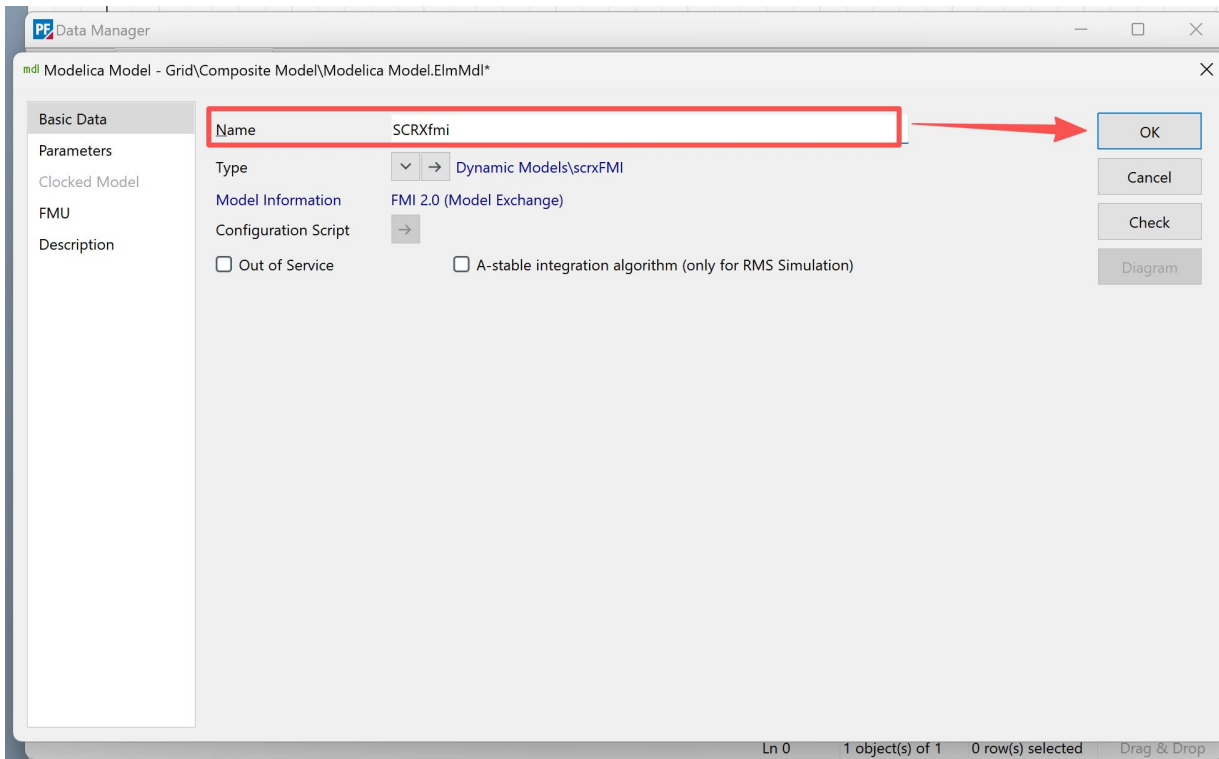
1. In the assignment dialog, navigate to the library path where the FMU-backed type was defined (here: *scrxFMI*).
2. Select the FMU-linked Modelica Model Type and click **OK**.

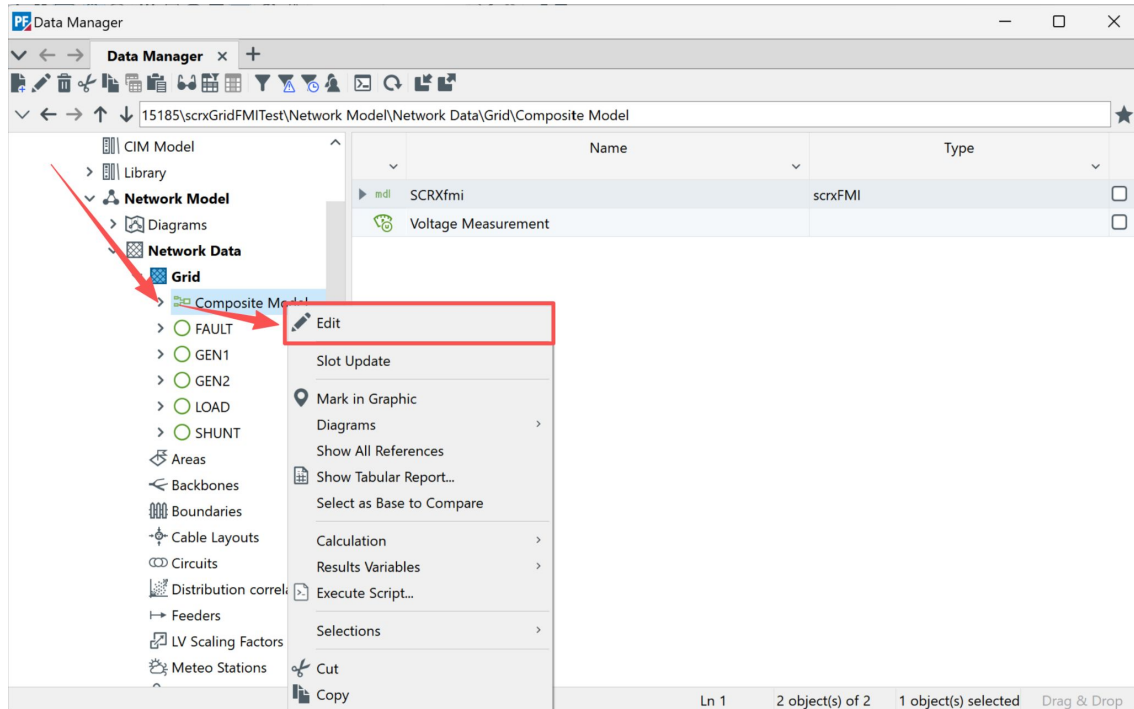
This step binds the network-level Modelica Model to the FMU you previously imported. With this link, the FMU becomes an active component inside the Composite Model and can exchange signals with the defined slots (ECOMP, AVR, GEN) during dynamic simulation.

In the **Modelica Model** properties, complete the setup of the FMU connection:

1. Enter a descriptive **Name** (e.g., *SCRXfmi*).
2. Ensure the **Type** points to the FMU-backed Modelica Model Type (e.g., *Dynamic Models\scrxFMI*).
3. Verify that the FMU information is correctly displayed (e.g., *FMI 2.0 – Model Exchange*).
4. Confirm settings and click **OK**.

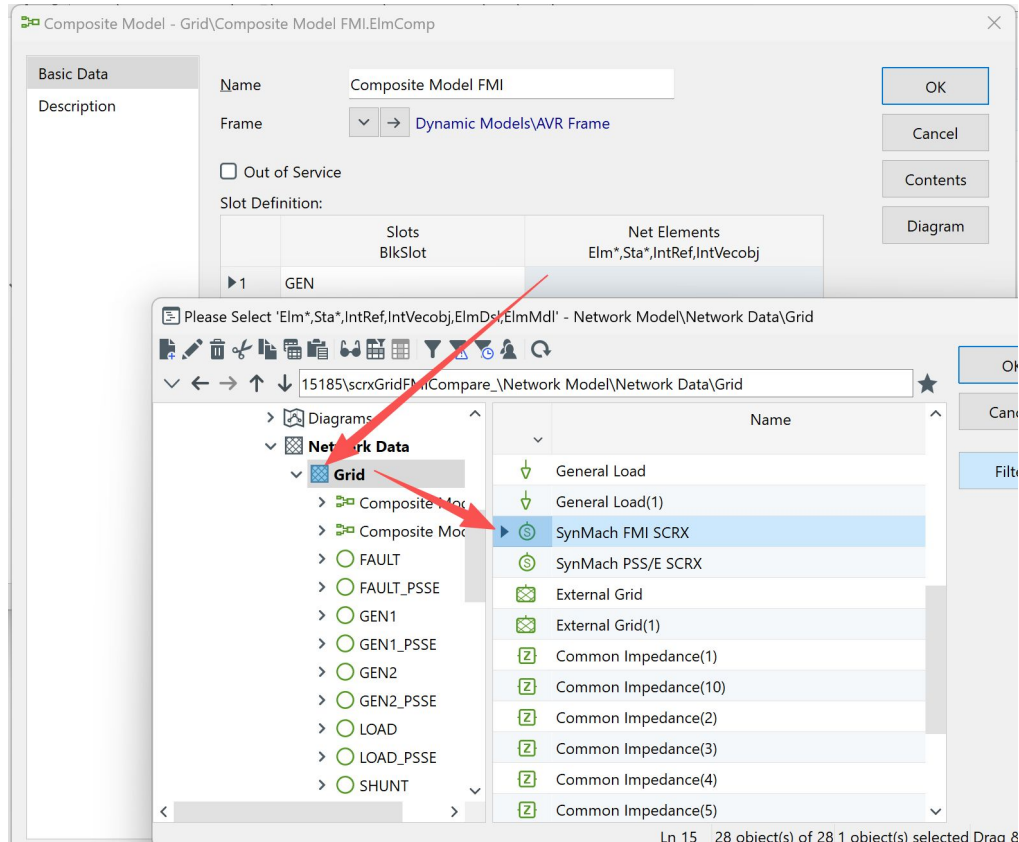
At this stage, the FMU is fully integrated into PowerFactory as a Modelica Model within the Composite Model. This completes the link from the FMU file → Model Type → Composite Frame → Network Model, enabling FMI-based co-simulation with the grid.



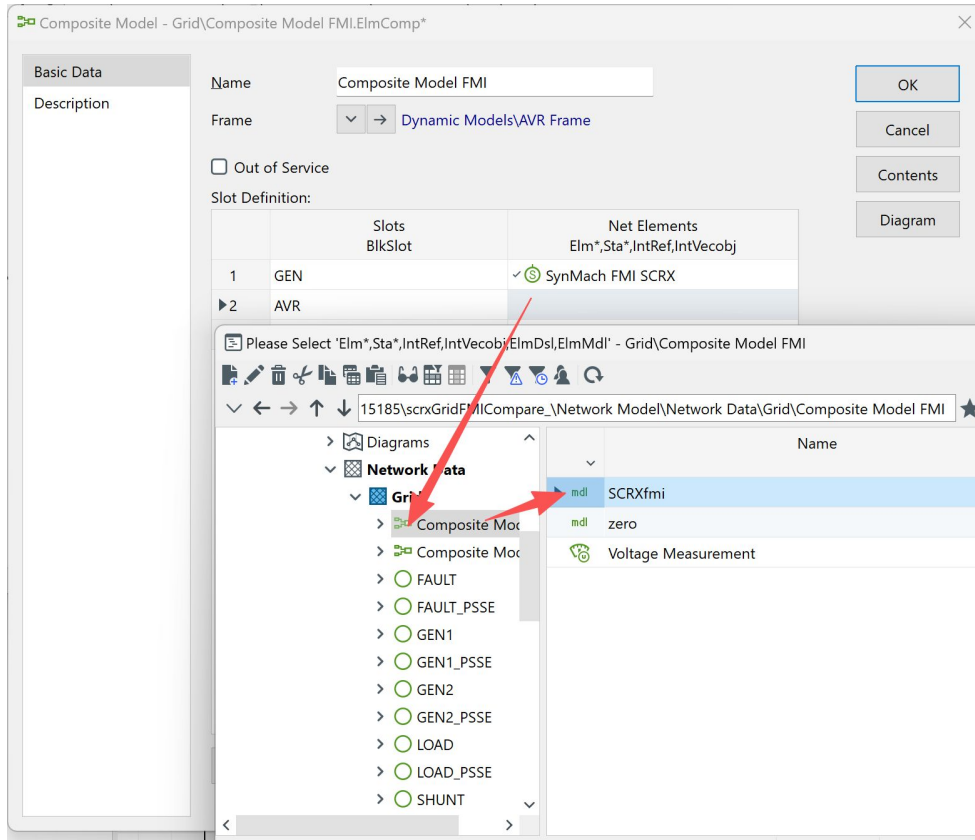


With the FMU and measurement objects added, the next step is to configure the Composite Model connections:

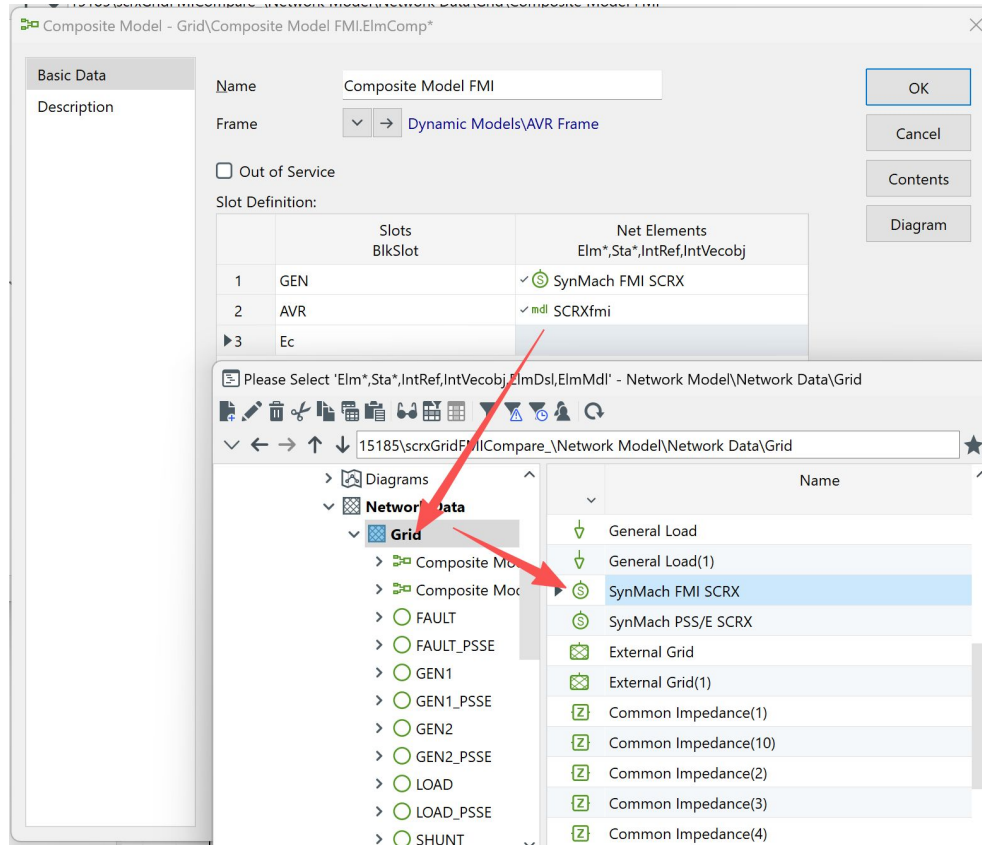
1. In the **Data Manager**, right-click on the **Composite Model**.
2. Select **Edit** from the context menu.



- Open the Composite Model**
In the *Network Data* window, locate the composite model where you want to integrate the FMU (for example, **Composite Model FMI**).
Double-click to open it.
- Assign the Model Frame**
In the *Basic Data* tab of the composite model, select the **Frame** field and choose the frame corresponding to your FMU-based Modelica model.
For instance, select **Dynamic Models\AVR Frame** or another appropriate dynamic model frame.
- Link the FMU-Based Machine**
In the slot definition section, assign your FMU-linked synchronous machine.
Navigate through the system hierarchy (as shown in the figure) and select **SynMach FMI SCR** from the list of available elements.



1. Click on the cell in the **Net Elements** column corresponding to the AVR slot.
2. In the selection dialog that appears, navigate to the location of your imported FMU (e.g., inside **Composite Model FMI**).
3. Choose **SCRXfmi** — the FMU representing the Modelica excitation system — and confirm your selection.



1. Click on the cell in the **Net Elements** column corresponding to the Ec slot.
2. In the selection dialog that appears, navigate to the location of your SynMach FMI SCR X.
3. Choose SynMach FMI SCR X and confirm your selection.

Composite Model - Grid\Composite Model FMI.ElmComp*

Basic Data

Description

Name: Composite Model FMI

Frame: ▼ → Dynamic Models\AVR Frame

☐ Out of Service

Slot Definition:

	Slots BlkSlot	Net Elements Elm*,Sta*,IntRef,IntVecobj
1	GEN	✓ SynMach FMI SCRX
2	AVR	✓ mdl SCRxfmi
▶3	Ec	✓ SynMach FMI SCRX

Slot Update

OK

Cancel

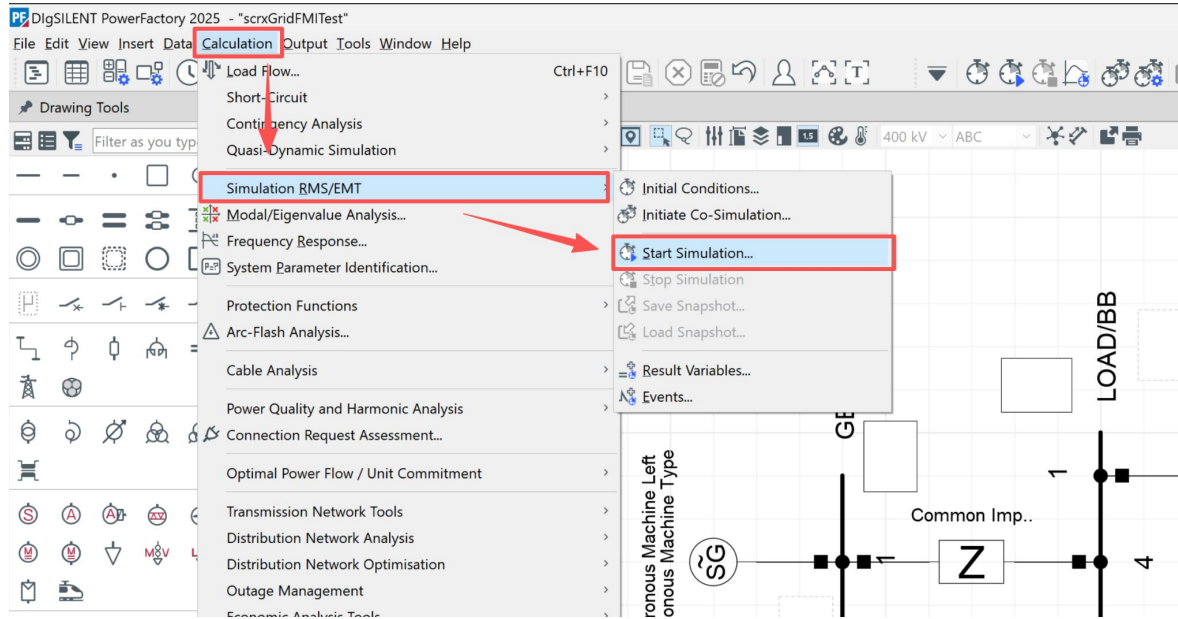
Contents

Diagram

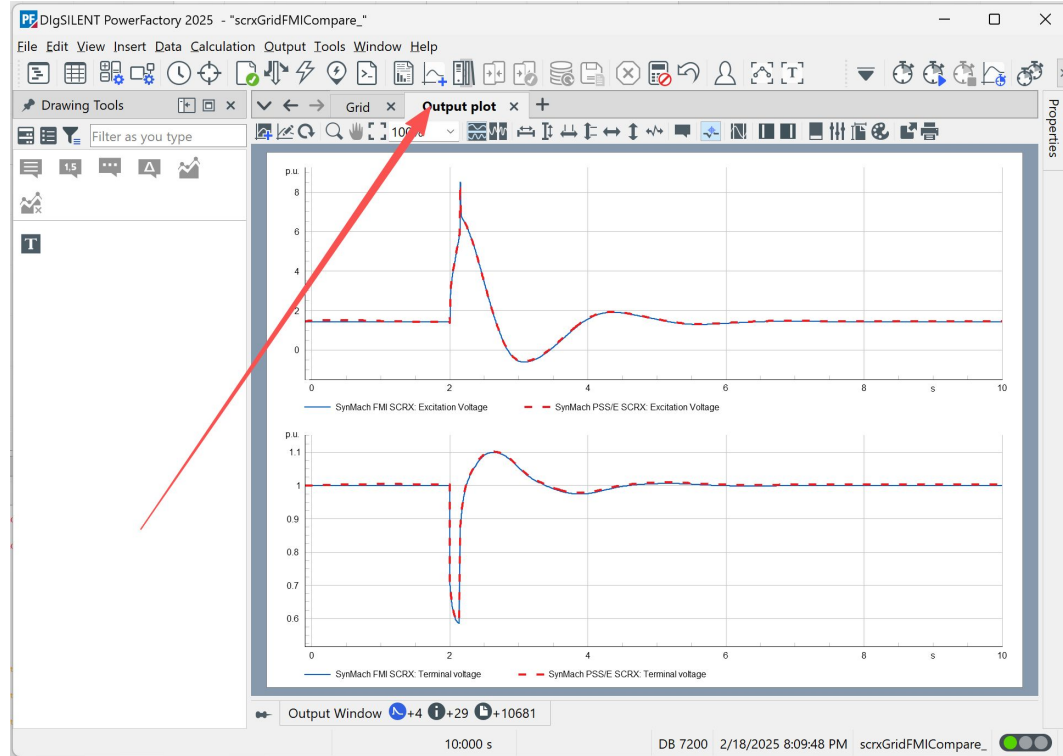
1. **GEN** → *Synchronous Machine Left*
2. **AVR** → *SCRXfmi* (imported FMU)
3. **Ec** → *Synchronous Machine Left*

At this point, the AVR frame is fully integrated into PowerFactory using the FMU exported from OpenModelica.

○



1. Go to the **Calculation** menu.
2. Select **Simulation RMS/EMT** → **Start Simulation**.
3. The simulation will run with the defined study case, recording the previously selected **excitation**.

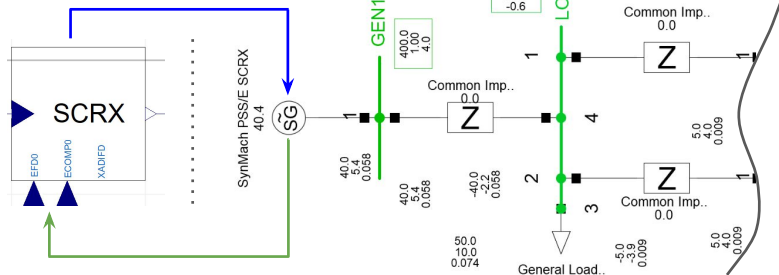


With this result, the tutorial is complete — you have successfully:

1. Exported an FMU from OpenModelica.
2. Imported and configured it in PowerFactory.
3. Built a composite AVR model.
4. Simulated and visualized the AVR's response.

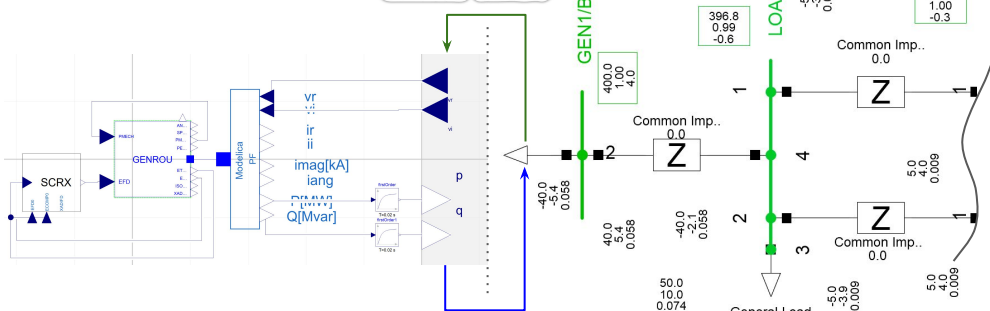
Powerfactory FMI Interfacing with OpenIPSL

Part II: Powerfactory OpenIPSL Interface Using FMI



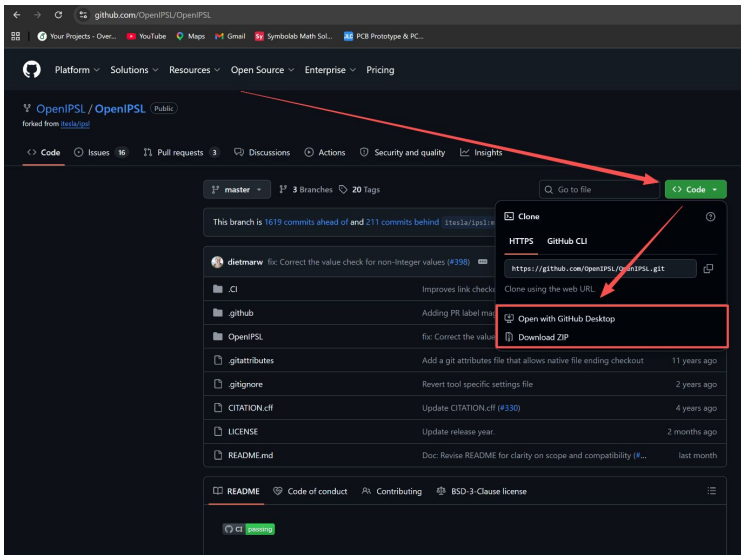
Top — Part I recap (controller-only FMI): Only the excitation system (SCRX) is imported as an FMU. The generator itself remains a native PowerFactory SynMach PSS/E SCR model, and the FMU exchanges just the scalar AVR signals (EFD0, ECOMP0, XADIFD) with the built-in machine. The rest of the network (GEN1_PS.., LOAD_PS.., common impedances, general load) is standard PowerFactory.

Bottom — Part II scope (full OpenIPSL machine via FMI): Both the controller and the synchronous machine are now Modelica models from the OpenIPSL library, packaged together and interfaced to PowerFactory through FMI. The FMU exposes the electrical interface of the machine directly — voltage (v_r , v_i), current (i_r , i_i , $i_{mag}[kA]$, i_{ang}), and power ($P[MW]$, $Q[Mvar]$) — which are tied into the PowerFactory grid at the GEN1/BB busbar through a Modelica-PF boundary block. Internally, the FMU contains the full GENROU machine model wrapped around the same SCRX excitation from Part I, with first-order filters smoothing the P/Q outputs before they are returned to the grid.



By the end of Part II you will understand:

- How to export this larger system as an FMU and bind it to a PowerFactory network through voltage/current boundary connections.
- Compare the OpenIPSL-backed generator response against PowerFactory's native PSS/E SynMach on the same grid, validating the co-simulation setup.



<https://github.com/OpenIPSL/OpenIPSL>

Before building the full Modelica generator system, you need a local copy of the **OpenIPSL** library. OpenIPSL (Open-Instance Power System Library) is an open-source Modelica library that provides validated power system component models — including the **GENROU** synchronous machine and **SCRX** excitation system used in this tutorial.

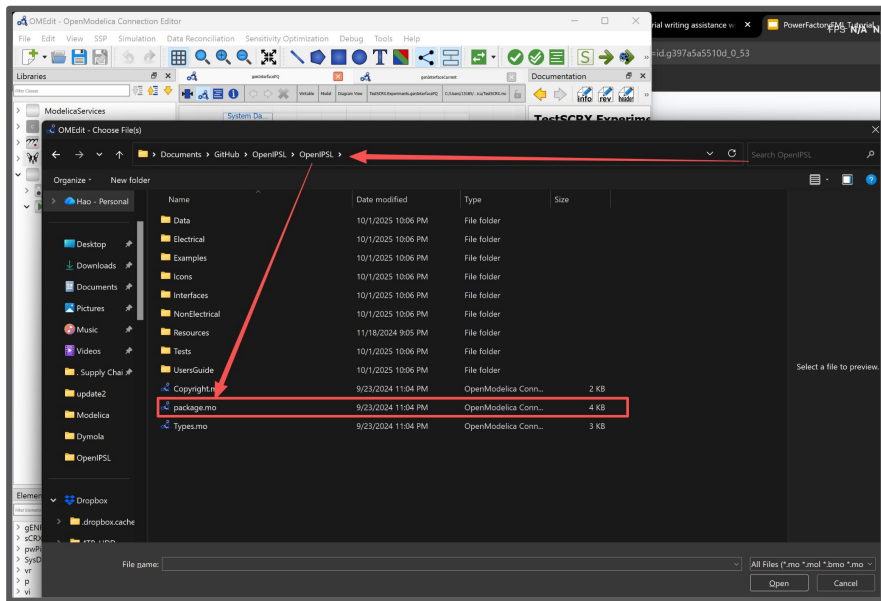
The library is hosted on GitHub under:

<https://github.com/OpenIPSL/OpenIPSL>

To get the files:

1. Navigate to the OpenIPSL repository page.
2. Click on the green **Code** button.
3. Choose one of the following options:
 - **Open with GitHub Desktop** (recommended if you use GitHub Desktop for version control).
 - **Download ZIP** (simplest option, extracts the library locally without Git).

Once downloaded, extract the archive to a known location on your machine. In the next steps, this **OpenIPSL** folder will be loaded into OMEdit as a Modelica library so that its component models — **GENROU**, **SCRX**, and the power-flow interface blocks — can be referenced when assembling the FMU for PowerFactory.

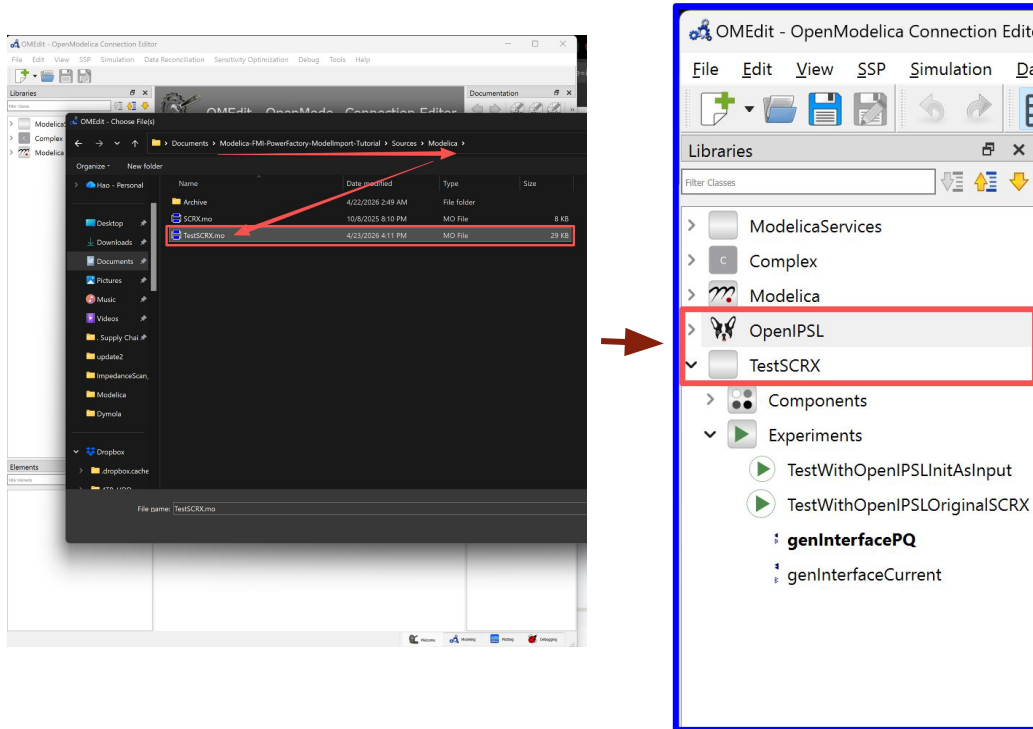


With the OpenIPSL repository downloaded, the next step is to load the library into OMEdit so its models become available in the workspace.

1. In OMEdit, go to **File** → **Open Model/Library File(s)** (or press **Ctrl+O**).
2. In the file browser, navigate to the cloned repository folder: **GitHub** → **OpenIPSL** → **OpenIPSL**.
3. Select the top-level **package.mo** file and click **Open**.

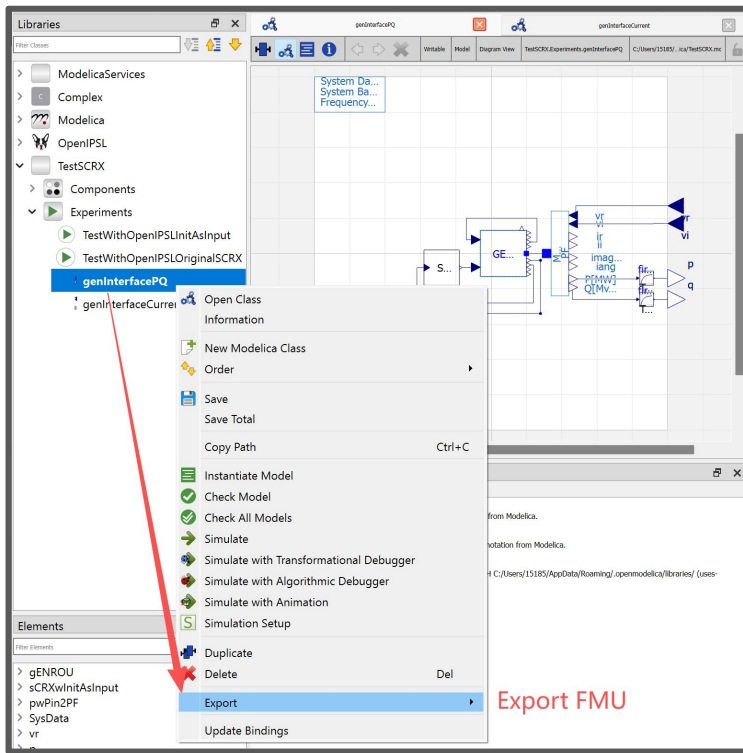
Loading **package.mo** (rather than an individual model file) is important — it registers the entire OpenIPSL library hierarchy with OMEdit, including subpackages such as **Electrical**, **Interfaces**, **NonElectrical**, and **Examples**. Once imported, OpenIPSL will appear in the **Libraries** panel on the left, and all of its components (e.g., **OpenIPSL.Electrical.Machines.PSSE.GENROU**, **OpenIPSL.Electrical.Controls.PSSE.ES.SCRX**) can be referenced in your own Modelica models.

This loaded library provides the building blocks used in the rest of Part II to assemble the full generator + AVR system that will be exported as an FMU.



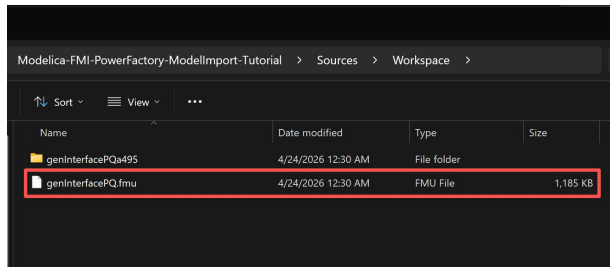
With OpenIPSL loaded, the next step is to bring in the tutorial's example package, which contains the pre-built generator + AVR test system used throughout Part II.

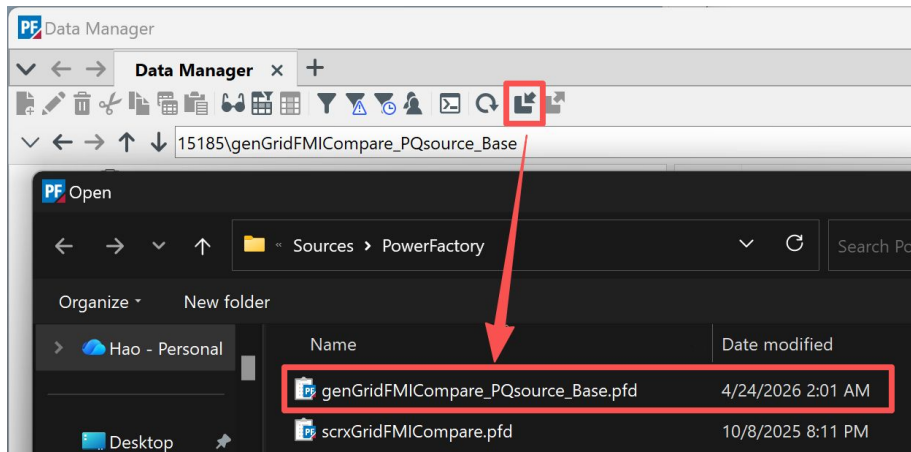
1. In OMEdit, go to **File** → **Open Model/Library File(s)** (or press **Ctrl+O**).
2. Navigate to the tutorial repository at [Modelica-FMI-PowerFactory-ModelImport-Tutorial](#) → [Sources](#) → [Modelica](#).
3. Select **TestSCRX.mo** and click **Open**.



With the **TestSCRX** package loaded, the **genInterfacePQ** model can now be exported as a Functional Mock-up Unit (FMU) for use in PowerFactory.

1. In the **Libraries** panel, expand **TestSCRX** → **Experiments** and locate **genInterfacePQ**.
2. Right-click on **genInterfacePQ** to open the context menu.
3. Select **Export** → **FMU**.



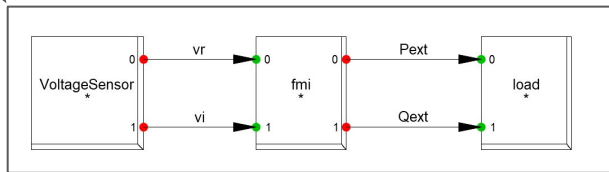
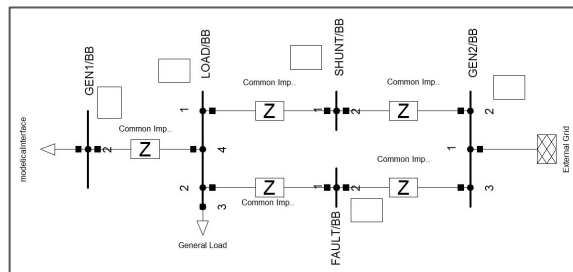
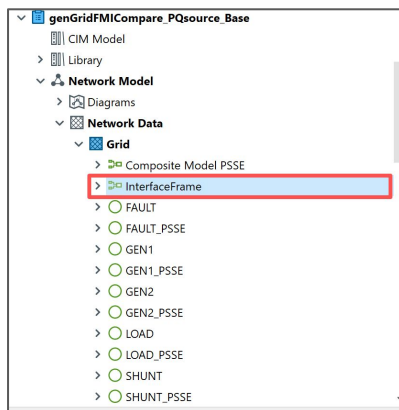


With the FMU exported, switch to PowerFactory and import the companion project that hosts the target grid and a pre-built interface frame for the full-machine FMU.

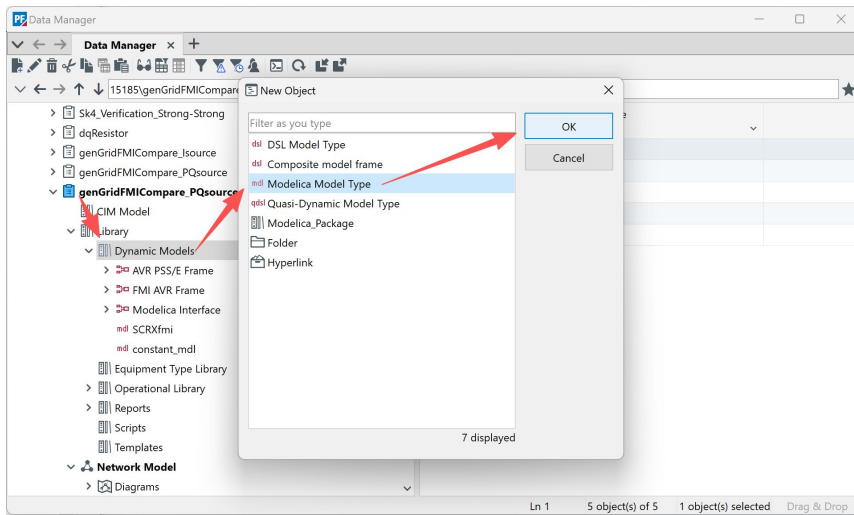
1. In PowerFactory's **Data Manager**, select or create a destination folder (here: **genGridFMICompare_PQsource_Base**) and click **Import Data**.
2. In the file browser, navigate to the tutorial's **Sources → PowerFactory** directory.
3. Select **genGridFMICompare_PQsource_Base.pfd** and click **Open**.
4. Confirm the options in the PFD Import dialog, click **Execute**, then right-click the imported project and choose **Activate**.

In the project tree, under **Network Model → Network Data → Grid**, the imported case already includes:

- **Composite Model PSSE** — a reference composite model using PowerFactory's native PSS/E **SynMach SCRXX**, which will serve as the baseline for validation.
- **InterfaceFrame** — a pre-built composite model frame (shown expanded in the bottom-right panel) that wires the FMU into the grid through a voltage/power boundary:
 - A **VoltageSensor** slot reads the terminal voltage at **GEN1/BB** and feeds **vr**, **vi** into the FMU.
 - The **fmi** slot is where the exported **genInterfacePQ.fmu** will be attached.
 - The FMU returns **Pext** and **Qext**, which drive a **load** slot that injects the machine's electrical output back into the grid at the same bus.



This **VoltageSensor** → **fmi** → **load** topology mirrors the Modelica-side **Modelica/PF** boundary block from the overview diagram: voltage flows into the FMU, power flows back out. In the next steps, the **genInterfacePQ.fmu** will be registered as a Modelica Model Type and bound to the **fmi** slot of this frame.

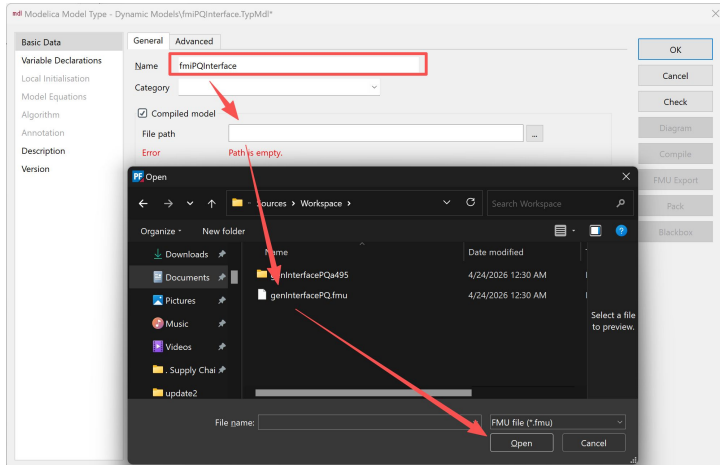


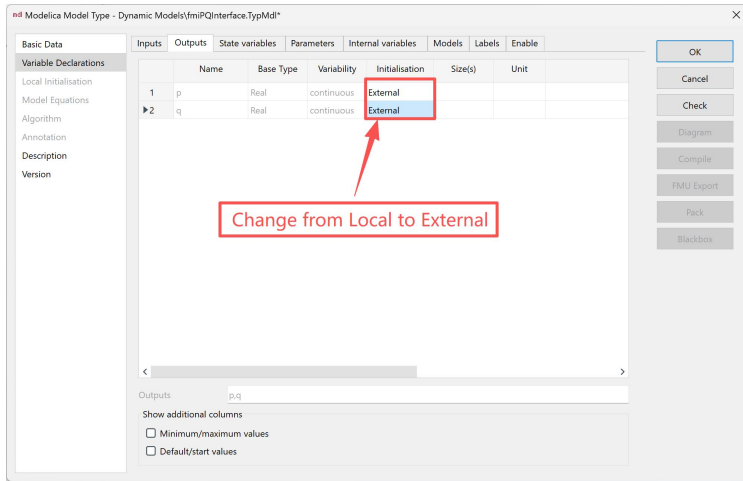
With the project imported, the next step is to register the exported `genInterfacePQ.fmu` inside PowerFactory's library so it can be referenced by the interface frame.

1. In the **Data Manager**, navigate to:
`genGridFMICompare_PQsource` → **Library** → **Dynamic Models**.
2. Click the **New Object** button in the toolbar.
3. In the **New Object** dialog, select **Modelica Model Type** and click **OK**.

In the **Modelica Model Type** properties window that opens:

1. Enter a descriptive **Name** — here, `fmiPQInterface`.
2. Check the **Compiled model** box to indicate this object references an external FMU rather than native Modelica code.
3. Click the browse button next to **File path**, navigate to **Sources** → **Workspace**, and select the `genInterfacePQ.fmu` file exported from OMEdit earlier.
4. Click **Open** to set the path, then confirm with **OK**.

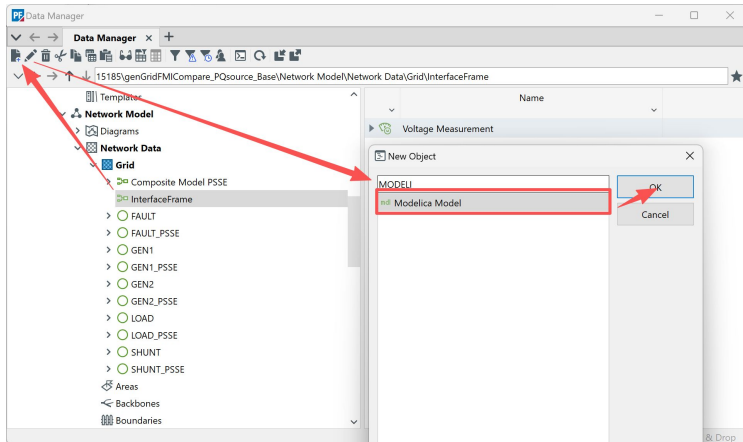




With the **fmiPQInterface** Model Type registered, two small but essential configuration steps remain: exposing the FMU's output variables to PowerFactory, and creating the Modelica Model instance inside the network's composite frame.

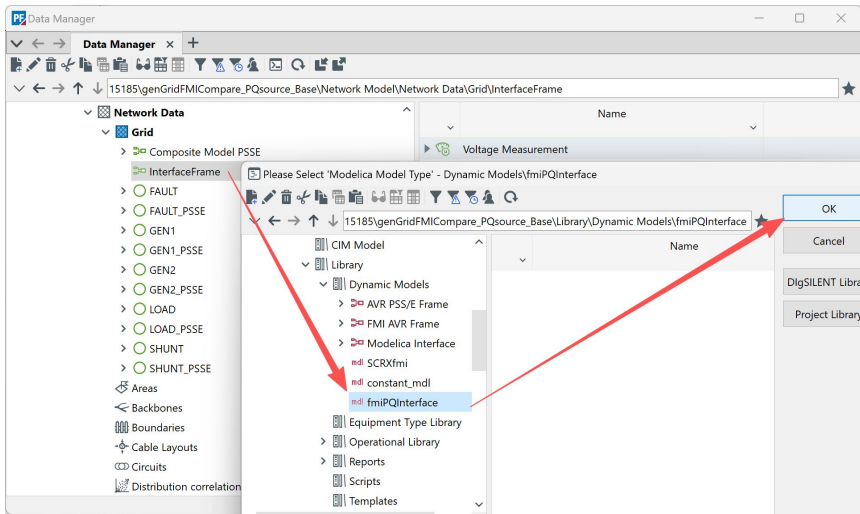
Top — Declare outputs as External:

1. Open the **fmiPQInterface** Model Type and select the **Variable Declarations** tab.
2. Switch to the **Outputs** sub-tab. The FMU's two electrical outputs, **p** and **q** (the active and reactive power injected back into the grid), are listed.
3. For both **p** and **q**, change the **Initialisation** column from **Local** to **External**.



Bottom — Instantiate the Modelica Model in the InterfaceFrame:

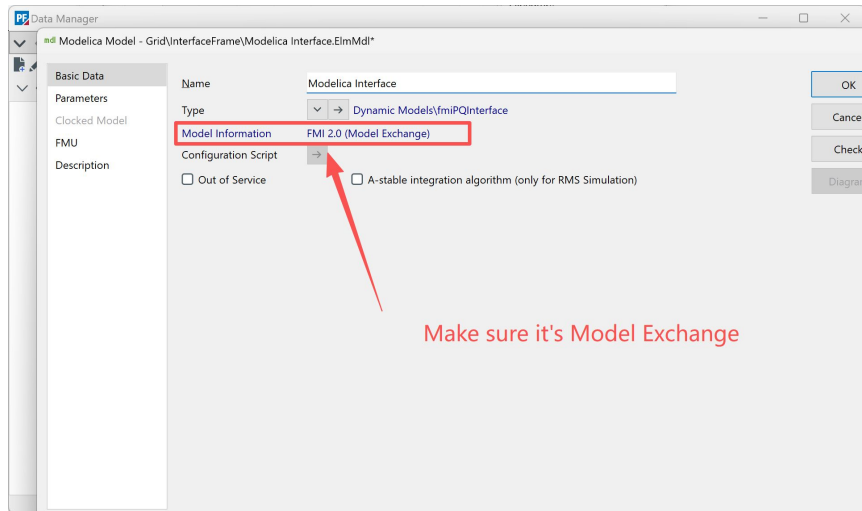
1. In the **Data Manager**, navigate to **Network Model** → **Network Data** → **Grid** → **InterfaceFrame**.
2. Click the **New Object** button in the toolbar.
3. In the **New Object** dialog, type **MODEL** into the filter, select **Modelica Model**, and click **OK**.



With the new Modelica Model instance created inside the **InterfaceFrame**, it now needs to be bound to the FMU-backed **fmiPQInterface** type registered earlier.

Top — Assign the Modelica Model Type:

1. In the **Data Manager**, navigate to **Network Model** → **Network Data** → **Grid** → **InterfaceFrame** and open the newly created Modelica Model.
2. In the assignment dialog, browse to **Library** → **Dynamic Models** and select **fmiPQInterface**.
3. Confirm with **OK**.

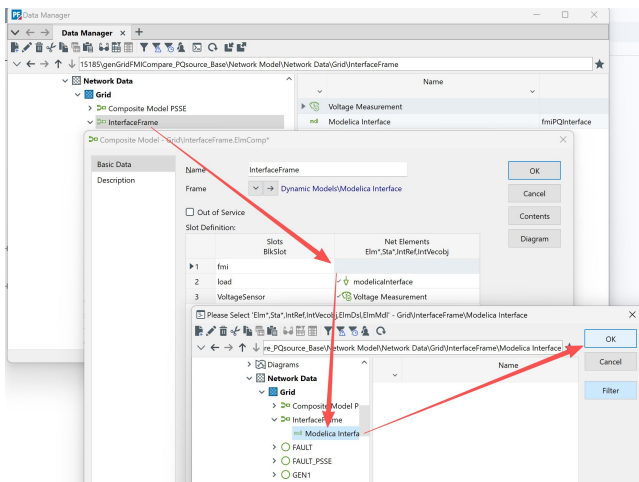


This binds the network-level Modelica Model to the compiled FMU (**genInterfacePQ.fmu**), so that during simulation the **InterfaceFrame**'s **fmi** slot is driven by the OpenIPSL-backed **GENROU + SCRX** system.

Bottom — Verify the Modelica Model properties:

In the Modelica Model properties window:

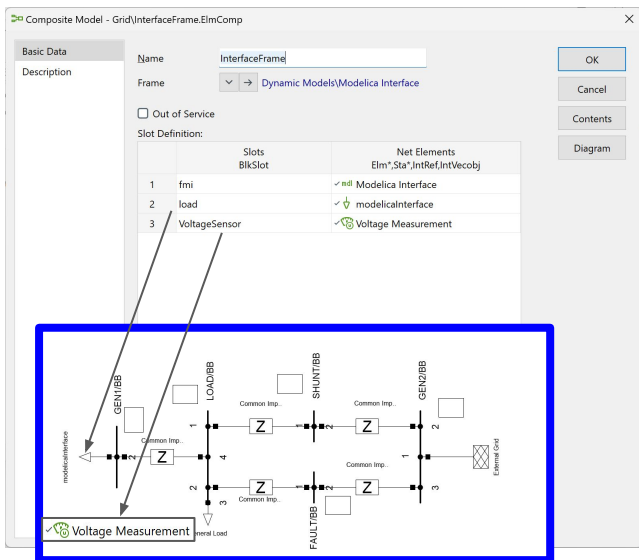
1. Set the **Name** (here: **Modelica Interface**).
2. Confirm that **Type** points to **Dynamic Models/fmiPQInterface**.



The final integration step is to map each slot in the **InterfaceFrame** composite model to a concrete network element, completing the data path between the FMU and the grid.

Top — Assign the **fmi** slot:

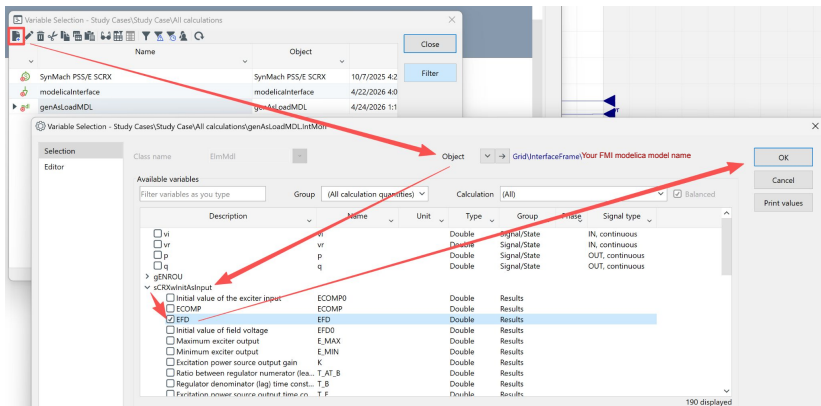
1. In the **Data Manager**, right-click **InterfaceFrame** and choose **Edit** to open its composite model properties.
2. In the **Slot Definition** table, click the **Net Elements** cell next to the **fmi** slot.
3. In the selection dialog, navigate to **Network Data** → **Grid** → **InterfaceFrame** and select **Modelica Interface** — the Modelica Model instance bound to **fmiPQInterface** in the previous step.
4. Confirm with **OK**.



Bottom — Completed slot mapping:

With all three slots assigned, the **InterfaceFrame** composite model is fully wired:

1. **fmi** → **Modelica Interface** — the OpenIPSL-backed FMU (**genInterfacePQ.fmu**) that contains the **GENROU** machine and **SCRX** excitation system.
2. **load** → **modelicaInterface** — the P/Q load element at the **GEN1/BB** busbar (visible on the left of the single-line diagram), which injects the FMU's **Pext** and **Qext** outputs back into the grid.
3. **VoltageSensor** → **Voltage Measurement** — the **StaVmea** element that measures the terminal voltage at **GEN1/BB** and feeds **vr**, **vi** into the FMU.



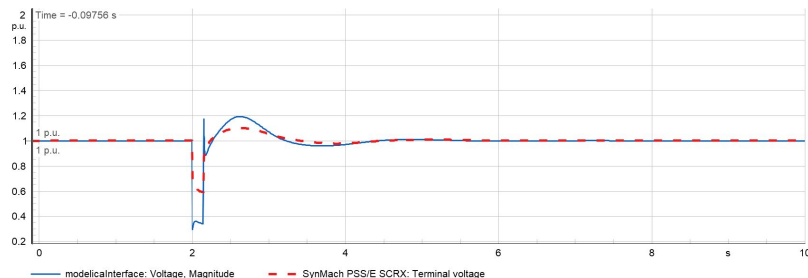
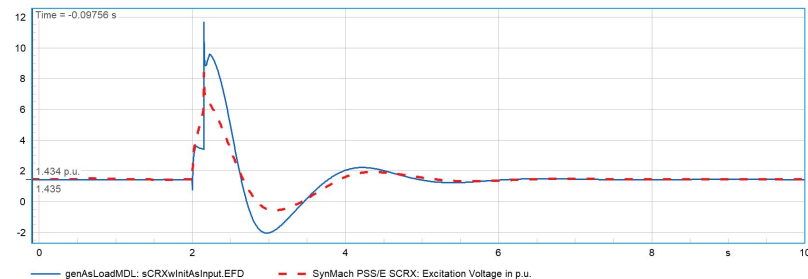
With the composite model fully wired, launch an RMS simulation and record the relevant signals to validate the OpenIPSL-backed generator against the native PSS/E reference.

Top — Selecting variables for recording:

1. From the **Study Case**, open **All calculations** → **Variable Selection** and choose the FMU-backed Modelica model (**Object** → **Grid\InterfaceFrame\Your FMI modelica model name**).
2. Expand the FMU's signal groups. Under **sCRXwInitAsInput**, enable **EFD** (the field voltage output of the OpenIPSL **SCRX**) and **ECOMP** (the compensated terminal voltage input). Expand **gENROU** to enable additional machine states as needed.
3. Repeat the process for the reference machine under **Composite Model PSSE**, selecting the corresponding **Excitation Voltage** and **Terminal voltage** signals on **SynMach PSS/E SCRX**.
4. Click **OK**, then run the simulation from **Calculation** → **Simulation RMS/EMT** → **Start Simulation**.

Bottom — Interpreting the comparison plots:

The two output plots overlay the FMU-backed response (solid blue) against the native PSS/E baseline (dashed red) during an excitation step / disturbance around $t = 2$ s:



- **Upper plot** — Field voltage **EFD**. The OpenIPSL **SCRX** reaches a noticeably higher peak (~9–10 p.u. vs. ~6 p.u.) before settling back to the same steady-state value (~1.43 p.u.) as the PSS/E reference.
- **Lower plot** — Terminal voltage magnitude. Both models dip to roughly the same minimum and recover to 1.0 p.u., but the FMU response shows a sharper transient right at the fault instant.